M.TECH · ORACLE DATABASE

PL/SQL (Oracle) — Visual Study Book

Basics → SQL & cursors → exceptions & subprograms → triggers & collections → dynamic SQL & a mini project. Every topic explained with a diagram, working example code with its output, an exam tip and a Hinglish memory hook.

Unit I · PL/SQL Fundamentals

Unit II · SQL in PL/SQL & Cursors

Unit III · Exceptions & Subprograms

Unit IV · Triggers, Collections, Bulk

Unit V · Dynamic SQL, Tuning, Project

32 original diagrams · worked examples with outputs · Hinglish memory hooks · revision checklist

Table of Contents

How to use this book ... 3

PL/SQL Fundamentals ... 5

1.1 What is PL/SQL? ... 5
1.2 PL/SQL architecture ... 6
1.3 Block structure ... 7
1.4 The sample schema and DBMS_OUTPUT ... 8
1.5 Variables, constants and datatypes ... 10
1.6 %TYPE and %ROWTYPE ... 11
1.7 Operators and NULL ... 12
1.8 Decision making: IF and CASE ... 12
1.9 Loops ... 14

SQL in PL/SQL and Cursors ... 17

2.1 Using SQL inside PL/SQL ... 17
2.2 What is a cursor? ... 18
2.3 Implicit cursor attributes ... 18
2.4 Explicit cursors ... 19
2.5 Cursor FOR loop ... 20
2.6 Parameterized cursors ... 21
2.7 FOR UPDATE and WHERE CURRENT OF ... 21
2.8 REF CURSOR (cursor variables) ... 22
2.9 Transactions: COMMIT, ROLLBACK, SAVEPOINT ... 23

Exceptions and Subprograms ... 26

3.1 Exception handling ... 26
3.2 Stored procedures ... 30
3.3 Stored functions ... 32
3.4 Packages ... 33
3.5 Managing stored code ... 35

Triggers, Collections and Bulk Processing ... 36

4.1 Triggers ... 36
4.2 Records and collections ... 40
4.3 BULK COLLECT and FORALL ... 43
4.4 Autonomous transactions ... 45

Dynamic SQL, Tuning and a Mini Project ... 46

5.1 Dynamic SQL ... 46
5.2 Definer rights and invoker rights ... 47
5.3 Performance and best practices ... 48
5.4 Mini project — bank money transfer ... 49
5.5 Classic practice programs ... 51
5.6 Quick viva answers ... 53

Quick comparison sheet ... 55

Exam-day revision checklist ... 57

Recommended resources ... 58

How to use this book

The 3-step exam answer formula

⭐ **Exam tip — Write every answer in this order**

1. **Definition** — one line (the blue 📘 boxes give you this ready-made).
2. **Explain in 3–5 lines** — plain English, use the keywords in **bold**.
3. **Diagram or code** — draw the figure from the book, or write the short example; for comparison questions finish with a table.

Box guide

Box	What it gives you
📘 Definition	Opening line for the answer
🏭 Real-life use case	Where this is used in real projects — good for viva questions
💡 Why it matters	The reason a feature exists (examiners love "why")
⭐ Exam tip	Marks-scoring points, what to memorise exactly
🧠 Yaad rakho	Short Hinglish memory hook
⚠️ Careful	Common mistakes, errors and traps
❓ Likely exam questions	Past-paper style questions for that unit

No notes PDF was attached for this topic, so this book follows the **standard Oracle PL/SQL syllabus** covered by most university courses. If your own syllabus lists extra topics (for example UTL_FILE, DBMS_SCHEDULER or object types), add them using the same structure.

How to run the examples

1. Install **Oracle Database Free** (or XE) and **SQL Developer**, or use a cloud / online Oracle SQL playground.
2. Connect as a normal user (not SYS). Run the table script in **Sec. 1.4** once to create the sample schema.
3. In SQL*Plus or SQL Developer's worksheet type `SET SERVEROUTPUT ON` first, then paste a block and end it with a **/** on its own line.
4. Examples that change data end with `ROLLBACK` so the sample rows stay the same for the next example.
5. If you see "Warning: compiled with errors", type `SHOW ERRORS` to find the line.

⚠️ **Careful**

The examples use features that need **Oracle 12c or newer** (identity columns, `FETCH FIRST`). Output lines such as `ORA-06512: at ...` contain your own schema name and line numbers, so yours will differ slightly.

Syllabus map

Unit	Focus	Main figures to practise
I	What PL/SQL is, architecture, block structure, variables, %TYPE, IF / CASE, loops	Block structure · engine · loops · IF flow
II	SQL inside PL/SQL, implicit and explicit cursors, cursor FOR loop, parameters, FOR UPDATE, REF CURSOR, transactions	Cursor life cycle · attributes · transaction timeline
III	Exceptions, procedures, functions, parameter modes, packages, stored code	Exception flow · parameter modes · package
IV	Triggers, :OLD / :NEW, records, collections, BULK COLLECT, FORALL, autonomous transactions	Trigger order · collections · bulk vs row-by-row
V	Dynamic SQL, SQL injection, invoker rights, tuning, bank-transfer project, practice programs	Static vs dynamic · transfer flow

The big picture — how the five units fit together

Unit I teaches the language; Unit II connects it to the data; Unit III packages logic into reusable, safe subprograms; Unit IV reacts to events and processes data in bulk; Unit V adds flexibility, speed and a complete project.

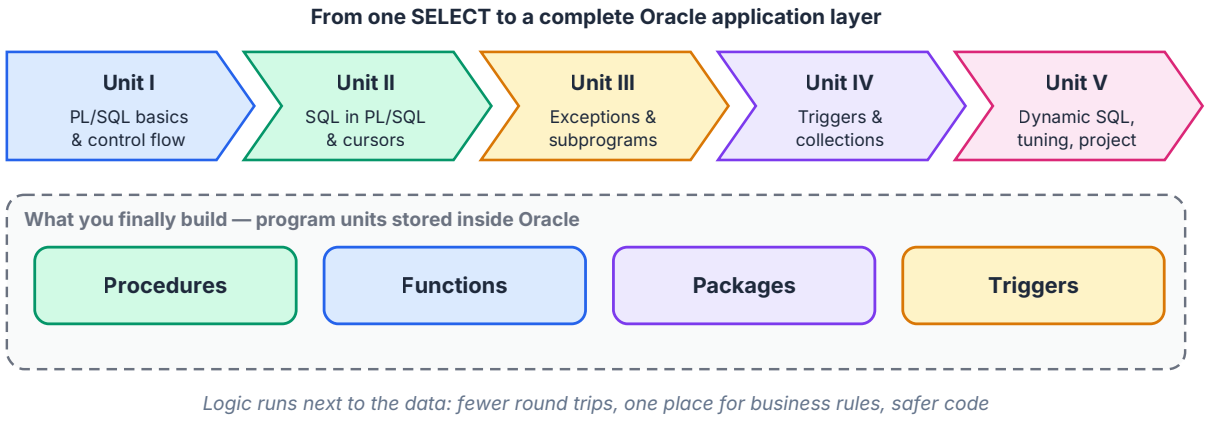


Fig. 1 — From one SELECT to a complete Oracle application layer

Real-life use case — One application, end to end

A banking app sends "transfer 15000 from account 1 to 2". A packaged procedure (Unit III) validates the request with exceptions, locks and updates rows through cursors and DML (Unit II), a trigger writes an audit row (Unit IV), and the whole thing runs in one atomic transaction that was written with the control flow of Unit I and tuned as in Unit V.

Practice reading: GeeksforGeeks PL/SQL hub — geeksforgeeks.org (PL/SQL) · Oracle documentation — docs.oracle.com (PL/SQL Language Reference) · Steven Feuerstein, *Oracle PL/SQL Programming* (O'Reilly)

UNIT I

PL/SQL Fundamentals

What & why | Architecture | Block structure | Variables & types | %TYPE / %ROWTYPE | IF / CASE | Loops

1.1 What is PL/SQL?

Definition

PL/SQL (Procedural Language extensions to SQL) is Oracle's block-structured programming language that lets you mix **SQL statements** with **procedural logic** — variables, conditions, loops, error handling — and store the program inside the database.

Plain SQL is **declarative**: you say *what* you want, one statement at a time. It has no variables, no loops and no `IF`. PL/SQL adds all of that on top of SQL, and sends a whole block of logic to the server in a **single round trip**.

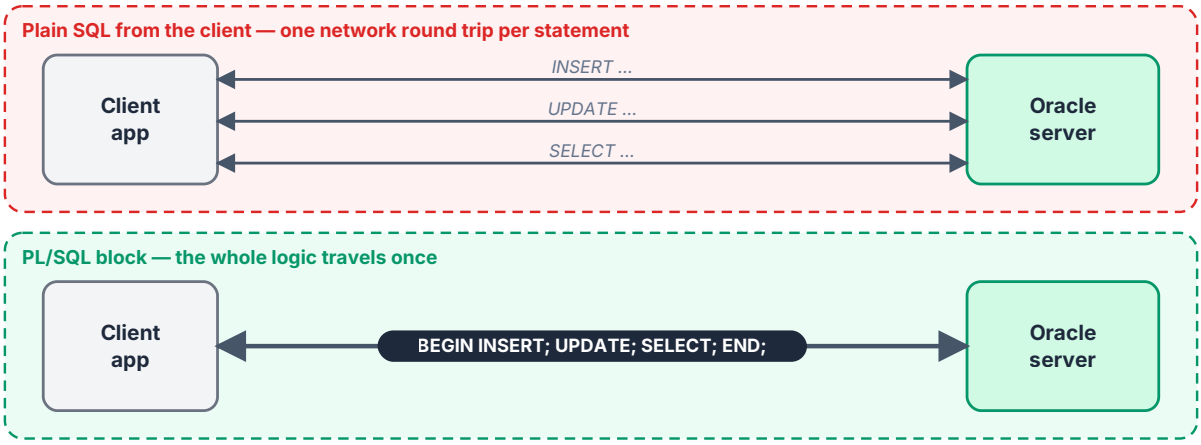


Fig. 2 — Plain SQL needs one network trip per statement; a PL/SQL block travels once

Point	SQL	PL/SQL
Nature	Declarative — what to do	Procedural — how to do it, step by step
Unit of execution	One statement	A whole block (many statements)
Variables, IF, loops	Not available	Fully available
Error handling	Error just stops the statement	EXCEPTION section to catch and handle errors
Stored in database	No (a query is just text)	Yes — procedures, functions, packages, triggers
Used for	Query, insert, update, delete, DDL	Business rules, batch jobs, validations, reports

Advantages of PL/SQL

- **Better performance** — a block is sent once; less network traffic.
- **Tight SQL integration** — SQL datatypes are PL/SQL datatypes; no conversion layer.

- **Modularity and reuse** — write once as a procedure or package, call from everywhere.
- **Error handling** — structured exception handling.
- **Portability and security** — runs on any Oracle platform; users can be given EXECUTE on a procedure without touching the tables.

🏭 Real-life use case — Where PL/SQL is used in real projects

A bank's "transfer money" rule, a payroll run that updates thousands of rows at month-end, an audit trigger that records who changed a salary, and the stored procedures that a Java or Flutter backend calls — all of these are PL/SQL living inside Oracle.

🧠 Yaad rakho

SQL = **kya chahiye** (what). PL/SQL = **kaise karna hai** (how) + SQL. Block ek baar jaata hai, isliye fast.

📖 GFG: PL/SQL Introduction

1.2 PL/SQL architecture

📘 Definition

The **PL/SQL engine** is the component inside the Oracle server (or inside tools like Oracle Forms) that compiles and runs PL/SQL blocks. It executes the **procedural** statements itself and hands every **SQL** statement to the **SQL statement executor**.

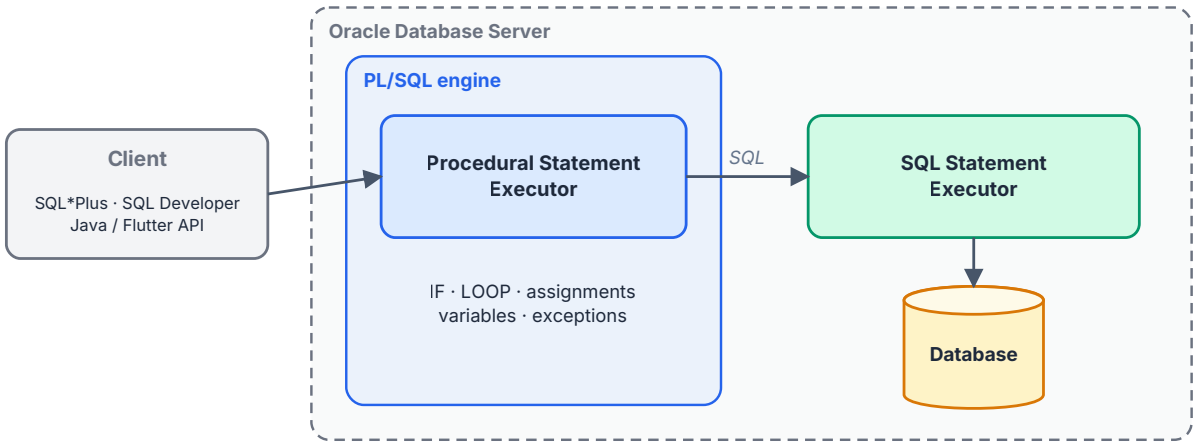


Fig. 3 — The PL/SQL engine runs procedural code and passes SQL to the SQL executor

1. The client (SQL*Plus, SQL Developer, an application) sends the block to the server.
2. The PL/SQL engine runs `IF`, loops and assignments by itself.
3. Whenever it meets a SQL statement it passes it to the SQL executor, which talks to the database.
4. The result comes back to the engine, which continues with the next statement.

⚠️ Careful — Context switch

Every hand-over between the PL/SQL engine and the SQL executor is a **context switch**. Thousands of switches inside a loop make code slow — this is why BULK COLLECT and FORALL exist (Unit IV).

1.3 Block structure

Definition

A **block** is the basic unit of PL/SQL. It has up to four sections: **DECLARE** (optional), **BEGIN** (mandatory), **EXCEPTION** (optional) and **END;**.

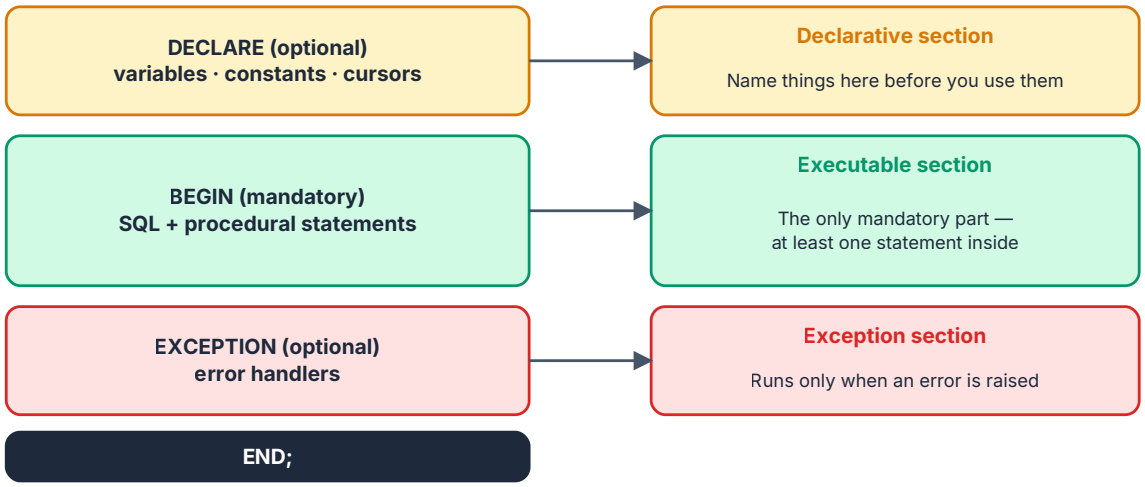


Fig. 4 — The four parts of a PL/SQL block

```sql
SET SERVEROUTPUT ON
DECLARE
  v_msg VARCHAR2(30) := 'Hello, PL/SQL!';
BEGIN
  DBMS_OUTPUT.PUT_LINE(v_msg);
END;
/
```

```text
Hello, PL/SQL!

PL/SQL procedure successfully completed.
```

In SQL*Plus and SQL Developer the **slash** `/` on its own line tells the tool to run the block. Every statement ends with a semicolon, and so does `END;`.

Types of blocks

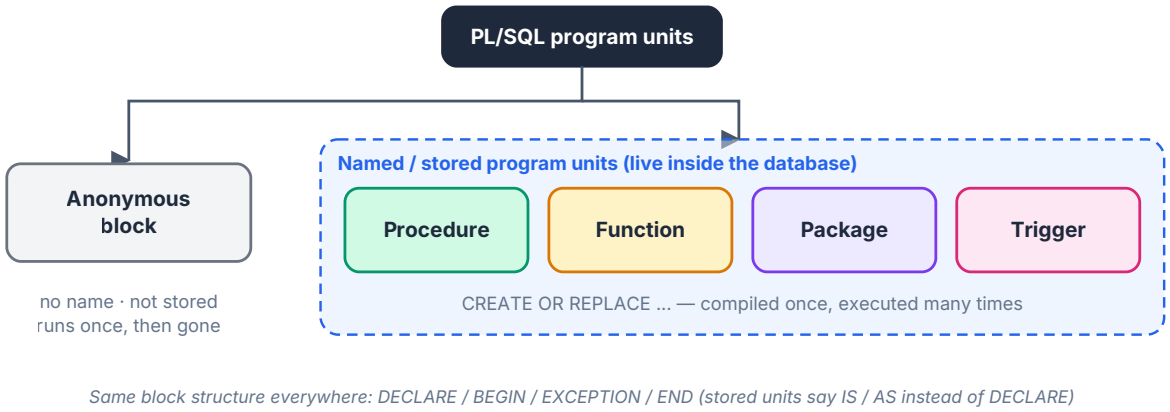


Fig. 5 — Anonymous blocks run once; named blocks are stored in the database

Nested blocks and scope

A block can contain another block. An inner block sees the variables of the outer block, but the outer block cannot see the inner block's variables.

```sql
DECLARE
  v_x NUMBER := 10;                -- outer variable
BEGIN
  DECLARE
    v_y NUMBER := 5;               -- inner variable
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Inner: x + y = ' || (v_x + v_y));
  END;
  DBMS_OUTPUT.PUT_LINE('Outer: x = ' || v_x);
  -- v_y is NOT visible here
END;
/
```

```text
Inner: x + y = 15
Outer: x = 10
```

⭐ **Exam tip**

Write the four parts with their **optional / mandatory** label — it is the most common 2-mark question. Only **BEGIN … END** is compulsory, and at least one executable statement must be inside it.

🧠 **Yaad rakho**

D-B-E-E: Declare, Begin, Exception, End. Sirf **Begin–End** zaroori hai.

1.4 The sample schema and DBMS_OUTPUT

Every example in this book uses the same two small tables, so you can type them once and try every program.

DEPARTMENTS

DEPT_ID	DEPT_NAME	LOCATION
10	IT	Nagpur
20	HR	Pune
30	Sales	Mumbai

FK

EMPLOYEES

EMP_ID	EMP_NAME	DEPT_ID	SALARY	HIRE_DATE
101	Amit	10	60000	15-JUN-21
102	Neha	10	45000	01-MAR-22
103	Ravi	20	35000	10-JAN-20
104	Sara	30	50000	25-AUG-23
105	Karan	10	28000	05-NOV-24

Sample schema used in every example of this book (EMP_ID and DEPT_ID are primary keys)

Fig. 6 — Sample schema — DEPARTMENTS and EMPLOYEES with a foreign key on DEPT_ID

```sql
CREATE TABLE departments (
  dept_id    NUMBER(4)    PRIMARY KEY,
  dept_name  VARCHAR2(30) NOT NULL,
  location   VARCHAR2(30)
);

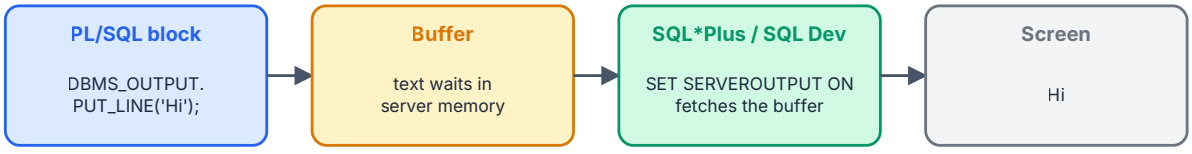
CREATE TABLE employees (
  emp_id     NUMBER(5)    PRIMARY KEY,
  emp_name   VARCHAR2(50) NOT NULL,
  dept_id    NUMBER(4)    REFERENCES departments (dept_id),
  salary     NUMBER(10,2),
  hire_date  DATE
);

INSERT INTO departments VALUES (10, 'IT',    'Nagpur');
INSERT INTO departments VALUES (20, 'HR',    'Pune');
INSERT INTO departments VALUES (30, 'Sales', 'Mumbai');

INSERT INTO employees VALUES (101, 'Amit',  10, 60000, DATE '2021-06-15');
INSERT INTO employees VALUES (102, 'Neha',  10, 45000, DATE '2022-03-01');
INSERT INTO employees VALUES (103, 'Ravi',  20, 35000, DATE '2020-01-10');
INSERT INTO employees VALUES (104, 'Sara',  30, 50000, DATE '2023-08-25');
INSERT INTO employees VALUES (105, 'Karan', 10, 28000, DATE '2024-11-05');
COMMIT;
```

Showing output: DBMS_OUTPUT

PL/SQL has no `print` statement. **DBMS_OUTPUT.PUT_LINE** writes text to a buffer on the server; the client shows it only when `SET SERVEROUTPUT ON` is active.



Without SET SERVEROUTPUT ON the text is stored but never shown

Fig. 7 — DBMS_OUTPUT puts text in a buffer; the client prints it when SERVEROUTPUT is ON

Use the concatenation operator `||` to join text and values. Numbers and dates are converted to text automatically, but a **BOOLEAN cannot be concatenated** — test it with `IF` instead.

1.5 Variables, constants and datatypes

📘 Definition

A **variable** is a named storage location whose value can change; a **constant** is declared with `CONSTANT` and must be given a value that can never change. Both are declared in the DECLARE section with a datatype.

```sql
variable_name [CONSTANT] datatype [NOT NULL] [:= value | DEFAULT value];
```

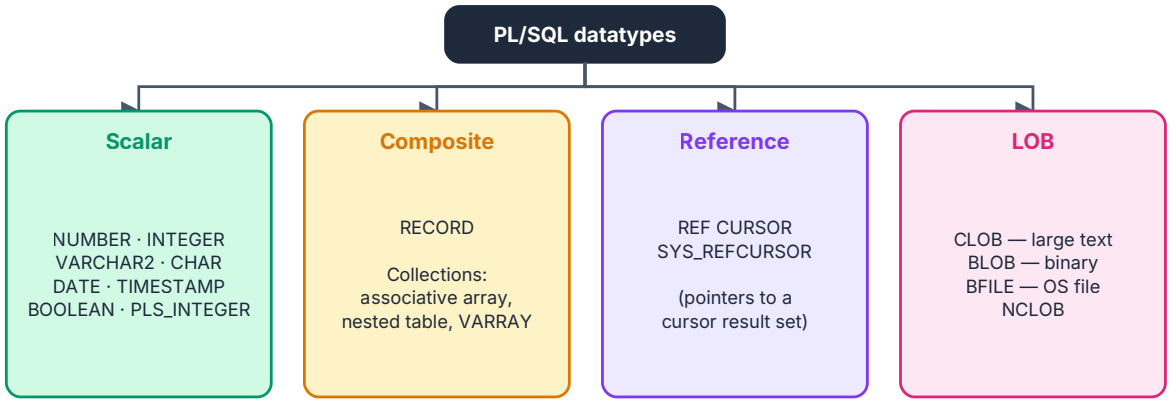


Fig. 8 — PL/SQL datatypes grouped into scalar, composite, reference and LOB

Datatype	Holds	Example
`NUMBER(p,s)`	Numbers with precision and scale	`price NUMBER(8,2) := 2500;`
`PLS_INTEGER`	Whole numbers (fast for counters)	`i PLS_INTEGER := 0;`
`VARCHAR2(n)`	Variable-length text up to n bytes	`name VARCHAR2(30);`
`CHAR(n)`	Fixed-length text (blank-padded)	`grade CHAR(1) := 'A';`
`DATE`	Date and time (to the second)	`d DATE := SYSDATE;`
`BOOLEAN`	TRUE, FALSE or NULL (in PL/SQL always; as a table column only from Oracle 23ai)	`ok BOOLEAN := TRUE;`

```sql
DECLARE
  c_gst      CONSTANT NUMBER(4,2) := 18;      -- GST percent, never changes
  v_price    NUMBER(8,2)          := 2500;
  v_name     VARCHAR2(20) DEFAULT 'Keyboard';
  v_total    NUMBER(10,2);
  v_in_stock BOOLEAN              := TRUE;
BEGIN
  v_total := v_price + (v_price * c_gst / 100);
  DBMS_OUTPUT.PUT_LINE(v_name || ' total = ' || v_total);
  IF v_in_stock THEN
    DBMS_OUTPUT.PUT_LINE('Available');
  END IF;
END;
/
```

```text
```

```
Keyboard total = 2950
Available
```

> ⚠️ **Careful**
>
> The assignment operator is `:=`, the comparison operator is a single `=`. A variable that is declared but not initialised holds **NULL**, and NULL plus anything is NULL.

1.6 %TYPE and %ROWTYPE

> 📘 **Definition**
>
> **%TYPE** declares a variable with the same datatype as a table column (or another variable). **%ROWTYPE** declares a record that has one field for every column of a table or cursor row.

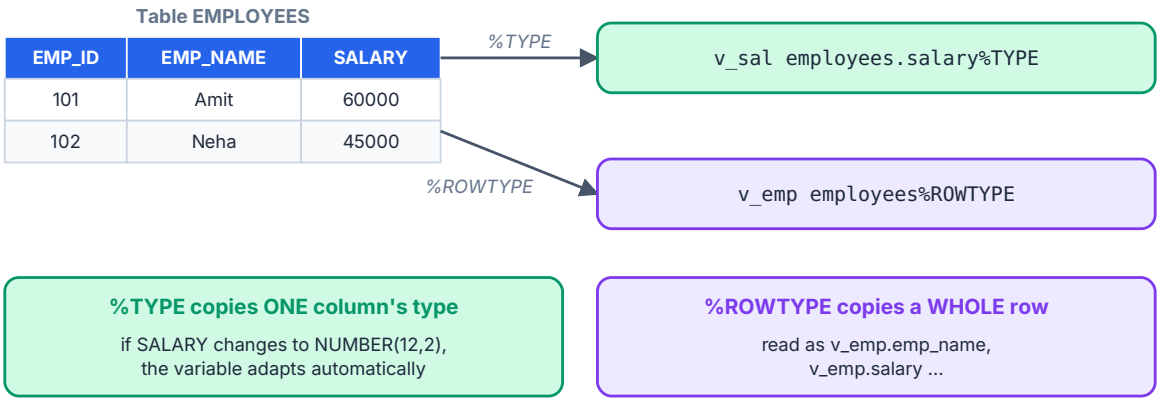


Fig. 9 — %TYPE copies one column; %ROWTYPE copies the whole row

```sql
DECLARE
  v_name  employees.emp_name%TYPE;
  v_sal   employees.salary%TYPE;
BEGIN
  SELECT emp_name, salary INTO v_name, v_sal
  FROM   employees
  WHERE  emp_id = 101;
  DBMS_OUTPUT.PUT_LINE(v_name || ' earns ' || v_sal);
END;
/
```

```text
Amit earns 60000
```

```sql
DECLARE
  v_emp employees%ROWTYPE;
BEGIN
  SELECT * INTO v_emp FROM employees WHERE emp_id = 102;
  DBMS_OUTPUT.PUT_LINE(v_emp.emp_name || ' | dept ' || v_emp.dept_id || ' | ' || v_emp.salary);
END;
/
```

```text
```

```
Neha | dept 10 | 45000
```

💡 **Why it matters**

If a DBA widens `salary` to `NUMBER(12,2)`, code written with `%TYPE` keeps working without any edit. Hard-coded types silently break or overflow. That is why professional PL/SQL uses `%TYPE` and `%ROWTYPE` everywhere.

🧠 **Yaad rakho**

%TYPE = ek column, **%ROWTYPE = poori row**. Table badle to code khud adjust ho jaata hai.

1.7 Operators and NULL

Kind	Operators
Arithmetic	`+ - * / **` (power); `MOD(a,b)` is a function, not an operator
Comparison	`= <> != < > <= >=`, `BETWEEN`, `IN`, `LIKE`, `IS NULL`
Logical	`AND OR NOT`
String	`||` concatenation
Assignment	`:=`

NULL means "unknown", not zero or blank. Comparing anything with NULL gives NULL (neither TRUE nor FALSE), so the following check never succeeds:

```sql
IF v_comm = NULL THEN ...      -- wrong: never TRUE
IF v_comm IS NULL THEN ...     -- right
v_total := NVL(v_comm, 0) + v_sal;   -- replace NULL with 0
```

Expression	Result	Why
`TRUE AND NULL`	NULL	Unknown whether the whole is true
`FALSE AND NULL`	FALSE	One FALSE settles AND
`TRUE OR NULL`	TRUE	One TRUE settles OR
`NOT NULL`	NULL	Not of unknown is unknown

1.8 Decision making: IF and CASE

📘 **Definition**

Conditional statements run a group of statements only when a condition is TRUE. PL/SQL offers `IF-THEN`, `IF-THEN-ELSE`, `IF-THEN-ELSIF` and the `CASE` statement / expression.

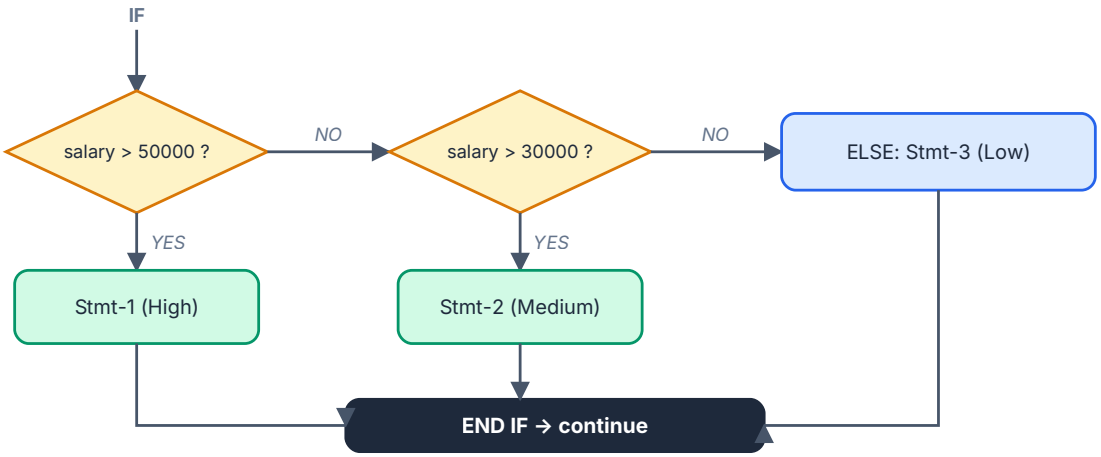


Fig. 10 — IF – ELSIF – ELSE: the first TRUE condition wins, then control joins at END IF

```sql
DECLARE
  v_sal   employees.salary%TYPE;
  v_bonus NUMBER;
BEGIN
  SELECT salary INTO v_sal FROM employees WHERE emp_id = 104;   -- Sara: 50000
  IF v_sal > 50000 THEN
    v_bonus := v_sal * 0.20;
  ELSIF v_sal > 30000 THEN
    v_bonus := v_sal * 0.10;
  ELSE
    v_bonus := v_sal * 0.05;
  END IF;
  DBMS_OUTPUT.PUT_LINE('Salary ' || v_sal || ' -> bonus ' || v_bonus);
END;
/
```

```text
Salary 50000 -> bonus 5000
```

Note the spelling **ELSIF** (not ELSEIF or ELSE IF) and that `END IF;` ends the whole chain.

CASE

```sql
DECLARE
  v_dept employees.dept_id%TYPE := 20;
  v_name VARCHAR2(20);
BEGIN
  v_name := CASE v_dept                -- simple CASE: compares one value
              WHEN 10 THEN 'IT'
              WHEN 20 THEN 'HR'
              WHEN 30 THEN 'Sales'
              ELSE 'Unknown'
            END;
  DBMS_OUTPUT.PUT_LINE('Department: ' || v_name);

  CASE                                 -- searched CASE: each WHEN is a condition
    WHEN v_dept = 10 THEN DBMS_OUTPUT.PUT_LINE('Tech team');
    WHEN v_dept = 20 THEN DBMS_OUTPUT.PUT_LINE('People team');
    ELSE                  DBMS_OUTPUT.PUT_LINE('Other team');
  END CASE;
END;
/
```

```text
Department: HR
People team
```

> ⚠️ **Careful**
>
> A `CASE` **statement** with no matching WHEN and no ELSE raises `CASE_NOT_FOUND` (ORA-06592). A CASE **expression** returns NULL in that situation. Always add an ELSE.

1.9 Loops

> 📘 **Definition**
>
> A **loop** repeats a group of statements. PL/SQL has three: the **basic LOOP** (exit tested inside), the **WHILE** loop (condition tested before each pass) and the **FOR** loop (counter managed automatically).

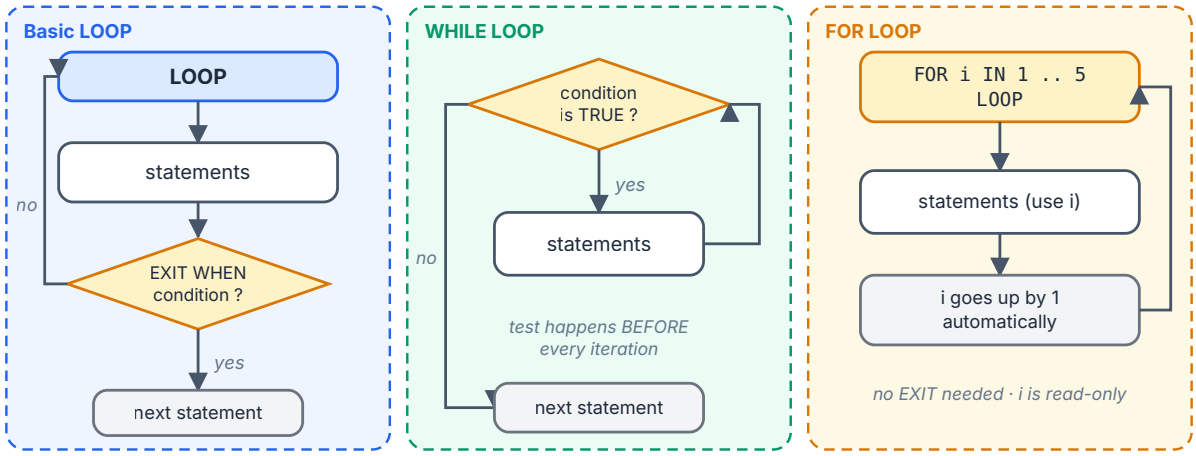


Fig. 11 — Basic LOOP, WHILE and FOR compared

Loop	Use when	Remember
Basic LOOP	Body must run at least once; exit decided in the middle	Forgetting `EXIT` gives an infinite loop
WHILE	Number of passes unknown; may run zero times	Change the variable inside the loop
FOR	Number of passes known	Counter is implicit and read-only; `REVERSE` counts down

```sql
-- Basic LOOP: sum of 1..5
DECLARE
  v_i   PLS_INTEGER := 1;
  v_sum PLS_INTEGER := 0;
BEGIN
  LOOP
    v_sum := v_sum + v_i;
    v_i   := v_i + 1;
    EXIT WHEN v_i > 5;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Sum 1..5 = ' || v_sum);
```

```
END;
/
```

```
TEXT
Sum 1..5 = 15
```

```
SQL
-- WHILE: factorial of 5
DECLARE
  v_n    NUMBER := 5;
  v_fact NUMBER := 1;
  v_i    NUMBER := 1;
BEGIN
  WHILE v_i <= v_n LOOP
    v_fact := v_fact * v_i;
    v_i    := v_i + 1;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_n || '! = ' || v_fact);
END;
/
```

```
TEXT
5! = 120
```

```
SQL
-- FOR with REVERSE, and CONTINUE
BEGIN
  FOR i IN REVERSE 1..3 LOOP
    DBMS_OUTPUT.PUT_LINE('Countdown: ' || i);
  END LOOP;

  FOR i IN 1..9 LOOP
    CONTINUE WHEN MOD(i, 2) = 0;        -- skip even numbers
    DBMS_OUTPUT.PUT_LINE('Odd: ' || i);
  END LOOP;
END;
/
```

```
TEXT
Countdown: 3
Countdown: 2
Countdown: 1
Odd: 1
Odd: 3
Odd: 5
Odd: 7
Odd: 9
```

Nested loops and labels

A label `<<name>>` lets `EXIT` leave an outer loop directly.

```
SQL
BEGIN
  <<outer>>
  FOR i IN 1..3 LOOP
    FOR j IN 1..3 LOOP
      EXIT outer WHEN i * j = 4;        -- leaves BOTH loops
      DBMS_OUTPUT.PUT_LINE(i || ' x ' || j || ' = ' || i * j);
    END LOOP;
  END LOOP outer;
```

```
END;
/
```

```
TEXT
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2
```

⭐ Exam tip

Questions like "differentiate between WHILE and FOR loops" are common. Give three points: **when the test happens, who controls the counter, and whether it can run zero times**. A FOR loop with `1..0` also runs zero times.

🧠 Yaad rakho

LOOP = pehle chalao, **exit andar**. WHILE = pehle **check**, phir chalao. FOR = **counter apne aap** badhta hai.

📖 GFG: PL/SQL control statements · PL/SQL loops

❓ Likely exam questions — Unit I

- What is PL/SQL? List its advantages over plain SQL.
- Explain the architecture of PL/SQL with a neat diagram.
- Describe the structure of a PL/SQL block. Which sections are optional?
- Explain `%TYPE` and `%ROWTYPE` with an example. Why are they useful?
- Write a PL/SQL block to find the factorial of a number.
- Differentiate between WHILE and FOR loops. Explain CONTINUE and labelled EXIT.
- What happens when you compare a value with NULL? Show the correct way.

UNIT II

SQL in PL/SQL and Cursors

SELECT INTO & DML | Implicit cursor | Explicit cursor | Cursor FOR loop | Parameters | FOR UPDATE | REF CURSOR | Transactions

2.1 Using SQL inside PL/SQL

Definition

PL/SQL supports **DML** (INSERT, UPDATE, DELETE, MERGE), **SELECT ... INTO** and **transaction control** (COMMIT, ROLLBACK, SAVEPOINT) directly. **DDL** (CREATE, ALTER, DROP) cannot be written directly — it needs dynamic SQL (Unit V).

`SELECT ... INTO` copies a query result into variables, but it must return **exactly one row**.

Rows returned	What happens
Exactly 1	Values copied into the variables — works
0 rows	Raises `NO_DATA_FOUND`
2 or more rows	Raises `TOO_MANY_ROWS` — use a cursor instead

```sql
DECLARE
  v_dept  departments.dept_name%TYPE;
BEGIN
  SELECT dept_name INTO v_dept FROM departments WHERE dept_id = 30;
  DBMS_OUTPUT.PUT_LINE('Dept 30 is ' || v_dept);

  INSERT INTO employees (emp_id, emp_name, dept_id, salary, hire_date)
  VALUES (106, 'Pooja', 30, 42000, SYSDATE);

  DELETE FROM employees WHERE emp_id = 106;
  ROLLBACK;                                  -- keep the sample data unchanged
END;
/
```

```text
Dept 30 is Sales
```

Exam tip

Variables and column names must not clash. If a variable is called `salary` and so is the column, the column wins inside SQL. Use a prefix: `v_salary`, `p_salary`.

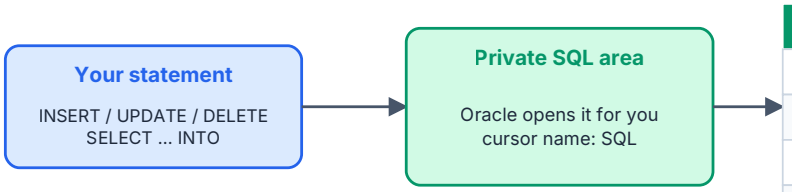
2.2 What is a cursor?

📘 Definition

A **cursor** is a pointer to a private memory area (the **context area**) where Oracle keeps a SQL statement and its result. PL/SQL uses cursors to process the rows returned by a query **one at a time**.

Kind	Who creates it	Used for
Implicit	Oracle, automatically	Every DML and every SELECT INTO
Explicit	You, in DECLARE	Queries returning **many rows**
REF CURSOR	You, at run time	Passing a result set to another program or client

2.3 Implicit cursor attributes



Attribute	Meaning
SQL%FOUND	1 or more rows hit
SQL%NOTFOUND	no row hit
SQL%ROWCOUNT	number of rows hit
SQL%ISOPEN	always FALSE

You never OPEN or CLOSE an implicit cursor — Oracle does it. Read the attributes right after the statement.

Fig. 12 — Oracle opens a cursor named SQL for every DML statement

```sql
BEGIN
  UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 10;
  DBMS_OUTPUT.PUT_LINE('Rows updated: ' || SQL%ROWCOUNT);
  IF SQL%NOTFOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employee in that department');
  END IF;
  ROLLBACK;
END;
/
```

```text
Rows updated: 3
```

Department 10 has Amit, Neha and Karan, so three rows change. The attributes always describe the **most recent** SQL statement, so read them straight after it.

⚠️ Careful

`SELECT INTO` with no row does not set `SQL%NOTFOUND` for you to test — it raises `NO_DATA_FOUND` first. Catch the exception instead (Unit III).

2.4 Explicit cursors

Definition

An **explicit cursor** is a named cursor that you declare, open, fetch from and close yourself. It is used when a query returns more than one row.

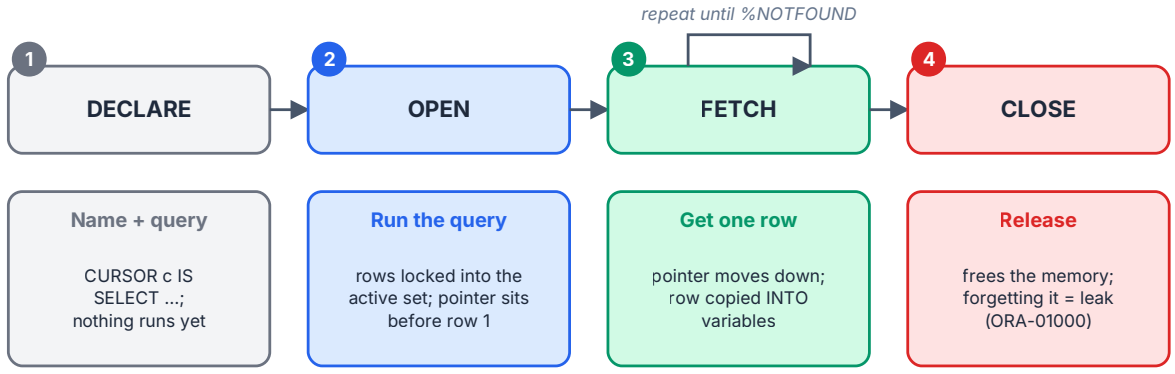


Fig. 13 — Four steps: DECLARE, OPEN, FETCH (repeated), CLOSE

```sql
CURSOR cursor_name IS select_statement;      -- DECLARE section
OPEN  cursor_name;
FETCH cursor_name INTO variable_list;        -- or INTO a %ROWTYPE record
CLOSE cursor_name;
```

```sql
DECLARE
  CURSOR c_emp IS
    SELECT emp_id, emp_name, salary
    FROM   employees
    WHERE  dept_id = 10
    ORDER  BY emp_id;
  v_id   employees.emp_id%TYPE;
  v_name employees.emp_name%TYPE;
  v_sal  employees.salary%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_id, v_name, v_sal;
    EXIT WHEN c_emp%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_id || '  ' || v_name || '  ' || v_sal);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Total rows: ' || c_emp%ROWCOUNT);
  CLOSE c_emp;
END;
/
```

```text
101  Amit  60000
102  Neha  45000
105  Karan  28000
Total rows: 3
```

Active set (WHERE dept_id = 10)

EMP_ID	EMP_NAME	SALARY
101	Amit	60000
102	Neha	45000
105	Karan	28000

Cursor attributes after each step

Step	%FOUND	%NOTFOUND	%ROWCOUNT
After OPEN	NULL	NULL	0
FETCH 1 (Amit)	TRUE	FALSE	1
FETCH 2 (Neha)	TRUE	FALSE	2
FETCH 3 (Karan)	TRUE	FALSE	3
FETCH 4 (no row)	FALSE	TRUE	3

EXIT WHEN c%NOTFOUND — put it right after FETCH, before you use the variables

Fig. 14 — The cursor attributes change after each FETCH

Attribute	Meaning
`c%FOUND`	TRUE if the last FETCH returned a row
`c%NOTFOUND`	TRUE if the last FETCH returned no row — the usual loop exit test
`c%ISOPEN`	TRUE while the cursor is open
`c%ROWCOUNT`	Number of rows fetched so far

⚠️ **Careful**

Put `EXIT WHEN c%NOTFOUND` **immediately after FETCH**. Putting it later makes the last row print twice. Always `CLOSE` the cursor — too many open cursors gives `ORA-01000`.

🧠 **Yaad rakho**

Cursor ke 4 kadam: **Declare, Open, Fetch, Close**. Attributes: **FOUND, NOTFOUND, ISOPEN, ROWCOUNT**.

2.5 Cursor FOR loop

📘 **Definition**

A **cursor FOR loop** opens the cursor, fetches every row into an implicitly declared record, and closes the cursor for you.

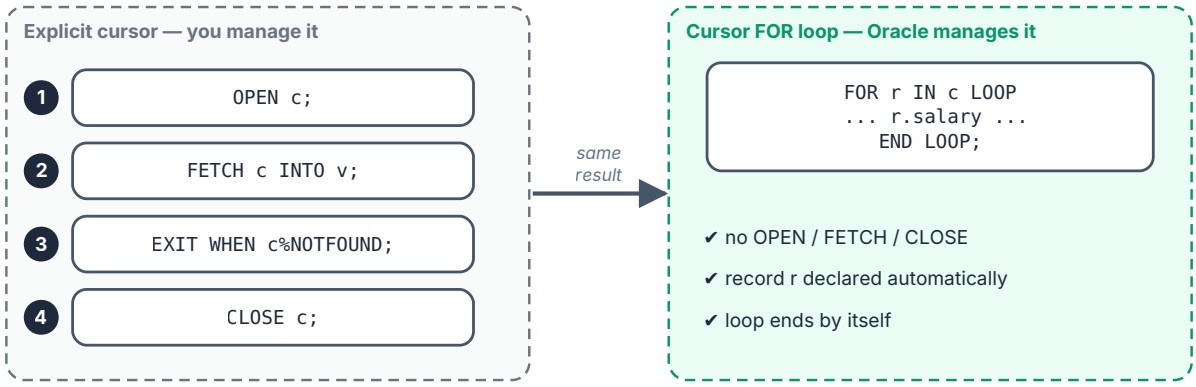


Fig. 15 — The cursor FOR loop does OPEN, FETCH, EXIT and CLOSE automatically

```sql
DECLARE
  v_total NUMBER := 0;
BEGIN
  FOR r IN (SELECT emp_name, salary
            FROM   employees
            WHERE  dept_id = 10
            ORDER  BY emp_id) LOOP
    DBMS_OUTPUT.PUT_LINE(r.emp_name || ': ' || r.salary);
    v_total := v_total + r.salary;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Dept 10 payroll = ' || v_total);
END;
/
```

```text
Amit: 60000
Neha: 45000
Karan: 28000
Dept 10 payroll = 133000
```

💡 **Why it matters**

Fewer lines, no forgotten CLOSE, no off-by-one with NOTFOUND. Use it as your **default** way to loop over query rows; use explicit OPEN / FETCH only when you need fine control (for example BULK COLLECT with LIMIT).

2.6 Parameterized cursors

A cursor can take parameters, so one definition serves many calls. A parameter may have a `DEFAULT`.

```sql
DECLARE
  CURSOR c_by_dept (p_dept NUMBER, p_min NUMBER DEFAULT 0) IS
    SELECT emp_name, salary
    FROM   employees
    WHERE  dept_id = p_dept AND salary >= p_min
    ORDER  BY emp_id;
BEGIN
  FOR r IN c_by_dept(10, 40000) LOOP
    DBMS_OUTPUT.PUT_LINE(r.emp_name || ' - ' || r.salary);
  END LOOP;
END;
/
```

```text
Amit - 60000
Neha - 45000
```

Karan (28000) is below the 40000 limit, so he is filtered out.

2.7 FOR UPDATE and WHERE CURRENT OF

📘 **Definition**

FOR UPDATE locks the rows of the cursor's query when it is opened. **WHERE CURRENT OF cursor** then updates or deletes exactly the row the cursor is positioned on — no need to repeat the key.

```sql
DECLARE
  CURSOR c_low IS
    SELECT emp_id, salary FROM employees
    WHERE  salary < 40000
    FOR UPDATE OF salary;
  v_cnt PLS_INTEGER := 0;
BEGIN
  FOR r IN c_low LOOP
    UPDATE employees SET salary = salary + 2000 WHERE CURRENT OF c_low;
    v_cnt := v_cnt + 1;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Rows raised: ' || v_cnt);
  ROLLBACK;                -- use COMMIT to keep the raise and release the locks
END;
/
```

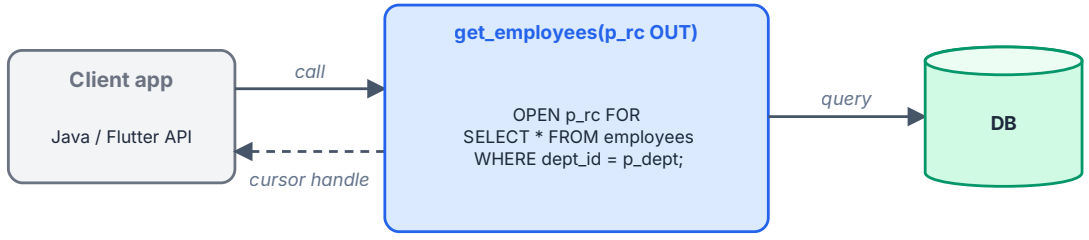
```text
Rows raised: 2
```

Ravi (35000) and Karan (28000) qualify. (The count is kept in a variable because a cursor FOR loop closes its cursor, so `c_low%ROWCOUNT` is no longer available after it.) The locks are held until `COMMIT` or `ROLLBACK`, so other sessions cannot change those rows meanwhile — use `FOR UPDATE NOWAIT` if you prefer an error to waiting.

> ⚠️ **Careful**
>
> Do not COMMIT inside a loop that fetches from a FOR UPDATE cursor: commit releases the locks and the next FETCH fails with ORA-01002 (fetch out of sequence).

2.8 REF CURSOR (cursor variables)

> 📘 **Definition**
>
> A **REF CURSOR** is a **variable** that points to a query result. Unlike a normal cursor, its query is chosen at run time with `OPEN ... FOR`, and it can be passed to and returned from subprograms.



The client fetches rows from the returned cursor — the result set is never copied into PL/SQL variables

Fig. 16 — A stored procedure returns a cursor handle; the client fetches the rows

```sql
CREATE OR REPLACE PROCEDURE get_dept_employees (
  p_dept IN  NUMBER,
  p_rc   OUT SYS_REFCURSOR
) AS
```

```
BEGIN
  OPEN p_rc FOR
    SELECT emp_id, emp_name, salary
    FROM   employees
    WHERE  dept_id = p_dept
    ORDER  BY emp_id;
END get_dept_employees;
/
```

```SQL
DECLARE
  v_rc   SYS_REFCURSOR;
  v_id   employees.emp_id%TYPE;
  v_name employees.emp_name%TYPE;
  v_sal  employees.salary%TYPE;
BEGIN
  get_dept_employees(20, v_rc);
  LOOP
    FETCH v_rc INTO v_id, v_name, v_sal;
    EXIT WHEN v_rc%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_id || '  ' || v_name || '  ' || v_sal);
  END LOOP;
  CLOSE v_rc;
END;
/
```

```TEXT
103  Ravi  35000
```

Kind	Declaration	Remark
Strong	`TYPE t IS REF CURSOR RETURN employees%ROWTYPE;`	Return type fixed; compiler checks it
Weak	`TYPE t IS REF CURSOR;`	Any query; checked at run time
SYS_REFCURSOR	`v SYS_REFCURSOR;`	Predefined weak type — what most projects use

🏭 **Real-life use case — How Java and Flutter backends use it**

A Java service calls the procedure through `CallableStatement`, registers the OUT parameter as `OracleTypes.CURSOR` and reads a normal `ResultSet`. This keeps the SQL inside the database and gives the app a stable interface.

2.9 Transactions: COMMIT, ROLLBACK, SAVEPOINT

📘 **Definition**

A **transaction** is a group of statements that must succeed or fail **together**. **COMMIT** makes the changes permanent, **ROLLBACK** undoes them, and **SAVEPOINT** marks a point to which you can roll back partially.

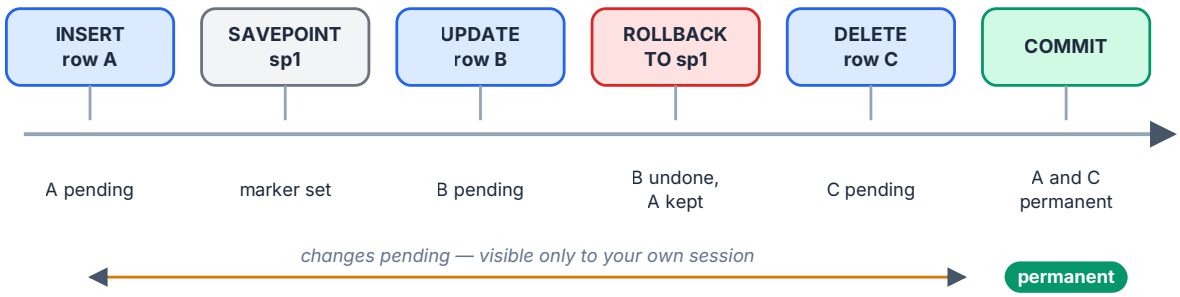


Fig. 17 — ROLLBACK TO a savepoint undoes only what came after it

```sql
BEGIN
  INSERT INTO departments VALUES (40, 'Finance', 'Delhi');
  SAVEPOINT sp1;
  UPDATE employees SET salary = salary + 1000 WHERE dept_id = 10;
  ROLLBACK TO sp1;          -- undoes only the UPDATE
  COMMIT;                   -- the new department is saved
END;
/
```

ACID	Meaning
Atomicity	All statements happen, or none
Consistency	Database moves from one valid state to another
Isolation	Other sessions do not see uncommitted changes
Durability	Committed data survives a crash

> ⚠️ **Careful**
>
> Any **DDL** statement (CREATE, ALTER, DROP) performs an **implicit COMMIT**. Also, a named subprogram should usually **not** commit by itself — let the caller decide when the whole business transaction is complete.

> 🧠 **Yaad rakho**
>
> COMMIT = **pakka**, ROLLBACK = **sab wapas**, SAVEPOINT = **beech ka bookmark**. DDL khud commit kar deta hai.

📖 GFG: PL/SQL cursors · COMMIT, ROLLBACK, SAVEPOINT

❓ Likely exam questions — Unit II

- What is a cursor? Differentiate between implicit and explicit cursors.
- Explain the four steps of using an explicit cursor with a diagram and a program.
- List and explain the cursor attributes.
- What is a cursor FOR loop? Why is it preferred?
- Write a parameterized cursor that lists employees of a given department above a salary.
- Explain FOR UPDATE and WHERE CURRENT OF with an example.
- What is a REF CURSOR? Differentiate strong and weak types.
- Explain COMMIT, ROLLBACK and SAVEPOINT. What are the ACID properties?

UNIT III

Exceptions and Subprograms

Exception handling | Predefined & user-defined | Procedures | Parameter modes | Functions | Packages | Stored code

3.1 Exception handling

Definition

An **exception** is a runtime error or warning condition. When one is raised, normal execution stops and control jumps to the **EXCEPTION section** of the block, where a matching **handler** decides what to do.

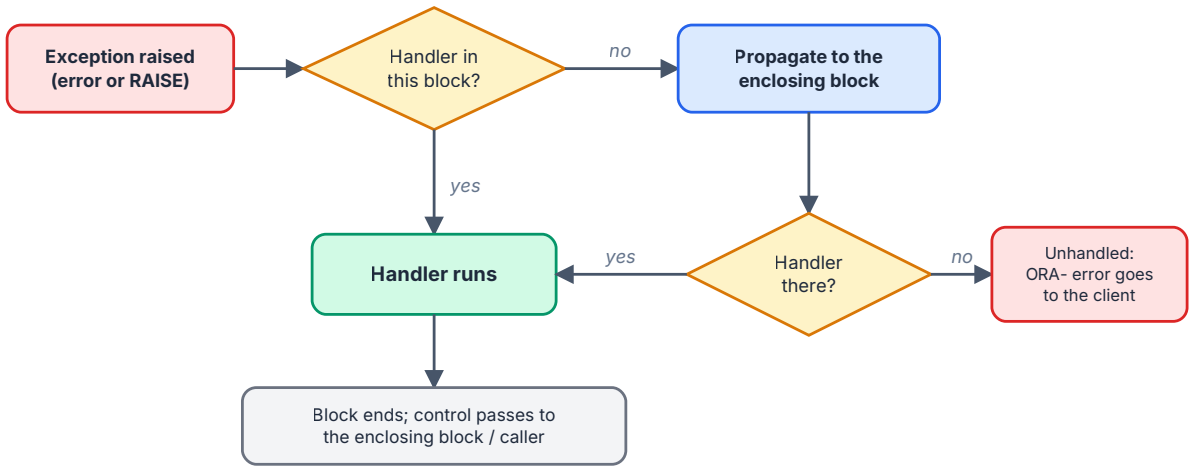


Fig. 18 — How an exception is looked up: this block first, then each enclosing block, then the caller

```sql
BEGIN
  ... statements ...
EXCEPTION
  WHEN exception_1 THEN  handler_statements;
  WHEN exception_2 OR exception_3 THEN  handler_statements;
  WHEN OTHERS THEN  handler_statements;      -- catches everything else, always last
END;
```

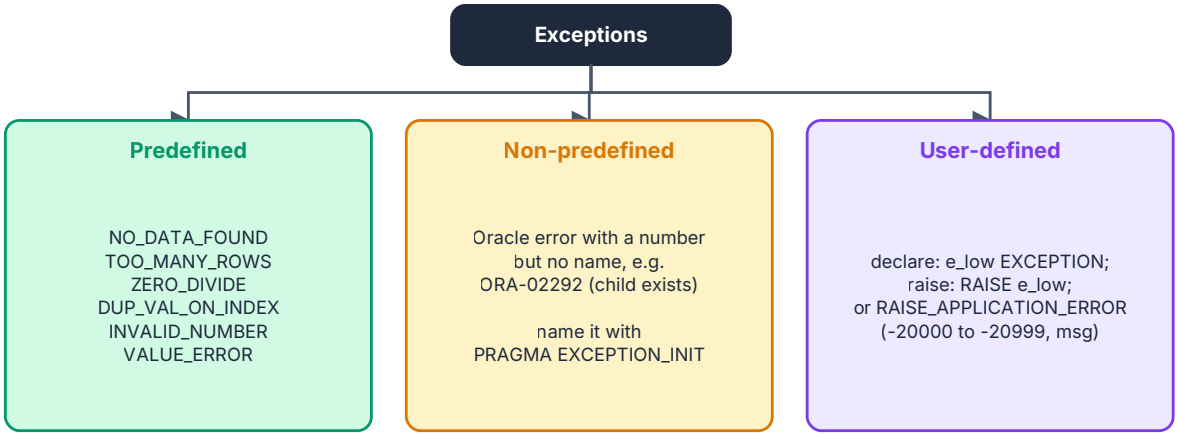


Fig. 19 — Three kinds of exceptions

Predefined exceptions

Oracle gives a name to the most common errors, so you can handle them without declaring anything.

Exception	Error	Raised when
`NO_DATA_FOUND`	ORA-01403	SELECT INTO returns no row
`TOO_MANY_ROWS`	ORA-01422	SELECT INTO returns more than one row
`ZERO_DIVIDE`	ORA-01476	Division by zero
`DUP_VAL_ON_INDEX`	ORA-00001	Duplicate value in a unique or primary key column
`INVALID_NUMBER`	ORA-01722	Text cannot be converted to a number in SQL
`VALUE_ERROR`	ORA-06502	Conversion, truncation or size error in PL/SQL
`INVALID_CURSOR`	ORA-01001	Illegal cursor operation, e.g. FETCH on a closed cursor
`CURSOR_ALREADY_OPEN`	ORA-06511	OPEN on a cursor that is already open

```sql
DECLARE
  v_name employees.emp_name%TYPE;
BEGIN
  SELECT emp_name INTO v_name FROM employees WHERE emp_id = 999;
  DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employee with id 999');
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('More than one employee found');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);
END;
/
```

```text
No employee with id 999
```

```sql
DECLARE
  v_zero NUMBER := 0;
BEGIN
```

```
  DBMS_OUTPUT.PUT_LINE(100 / v_zero);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);
END;
/
```

```
TEXT
Error -1476: ORA-01476: divisor is equal to zero
```

SQLCODE returns the error number (a negative number; `+100` for NO_DATA_FOUND) and **SQLERRM** returns its message. Store them in variables first if you want to use them inside a SQL statement.

User-defined exceptions

Declare it, raise it with `RAISE` when your own rule is broken, and handle it by name.

```
SQL
DECLARE
  e_low_salary EXCEPTION;
  v_sal        employees.salary%TYPE;
BEGIN
  SELECT salary INTO v_sal FROM employees WHERE emp_id = 105;   -- Karan: 28000
  IF v_sal < 30000 THEN
    RAISE e_low_salary;
  END IF;
  DBMS_OUTPUT.PUT_LINE('Salary OK');
EXCEPTION
  WHEN e_low_salary THEN
    DBMS_OUTPUT.PUT_LINE('Salary below minimum: ' || v_sal);
END;
/
```

```
TEXT
Salary below minimum: 28000
```

Non-predefined errors: PRAGMA EXCEPTION_INIT

Some Oracle errors have a number but no name. `PRAGMA EXCEPTION_INIT` attaches a name of your choice to that error number.

```
SQL
DECLARE
  e_child_exists EXCEPTION;
  PRAGMA EXCEPTION_INIT (e_child_exists, -2292);     -- ORA-02292: child record found
BEGIN
  DELETE FROM departments WHERE dept_id = 10;        -- IT still has employees
EXCEPTION
  WHEN e_child_exists THEN
    DBMS_OUTPUT.PUT_LINE('Cannot delete: employees still belong to this department');
END;
/
```

```
TEXT
Cannot delete: employees still belong to this department
```

RAISE_APPLICATION_ERROR

📘 **Definition**

RAISE_APPLICATION_ERROR(error_number, message) stops the program and returns your own error to the caller. The number must be between **-20000 and -20999**; the message may be up to 2048 bytes.

```text
ORA-20001: Percent must be positive
ORA-06512: at "YOURUSER.RAISE_SALARY", line 5
```

This is the standard way for a stored procedure to tell a Java or Flutter backend *why* a request was refused (see the `raise_salary` procedure in 3.2).

Propagation through nested blocks

```sql
DECLARE
  v_zero NUMBER := 0;
BEGIN
  BEGIN
    DBMS_OUTPUT.PUT_LINE(10 / v_zero);
  EXCEPTION
    WHEN NO_DATA_FOUND THEN                          -- does not match ZERO_DIVIDE
      DBMS_OUTPUT.PUT_LINE('inner: no data');
  END;
  DBMS_OUTPUT.PUT_LINE('skipped: the error is still active');
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('outer caught ZERO_DIVIDE');
END;
/
```

```text
outer caught ZERO_DIVIDE
```

⚠️ **Careful**

After a handler finishes, the block ends — control does **not** go back to the statement that failed. An exception raised in the **DECLARE** section (for example a bad initial value) is not caught by the same block's handlers; only the enclosing block can catch it.

⭐ **Exam tip**

Good practice: handle specific exceptions first, keep `WHEN OTHERS` last, and never leave it empty with `NULL;` — that hides real bugs. Log the error or re-raise it with `RAISE;`.

🧠 **Yaad rakho**

Teen type: **Predefined** (naam mila hua), **Non-predefined** (number hai, naam do via PRAGMA), **User-defined** (khud RAISE karo). **OTHERS hamesha aakhri.**

3.2 Stored procedures

Definition

A **procedure** is a named PL/SQL block, stored in the database, that performs an action. It can take parameters and is called as a statement.

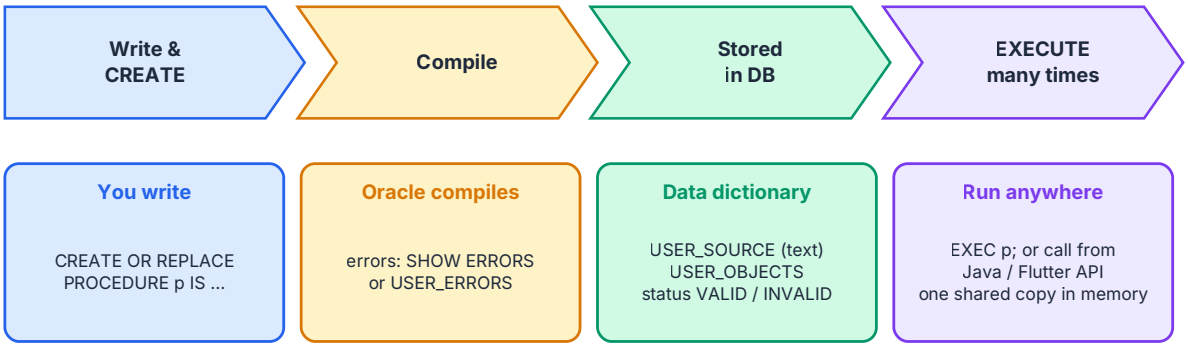


Fig. 20 — From CREATE to repeated execution — compiled once, stored in the database

```sql
CREATE [OR REPLACE] PROCEDURE name [(parameter [IN | OUT | IN OUT] datatype [DEFAULT value], ...)]
IS | AS
  -- declarations
BEGIN
  -- statements
EXCEPTION
  -- handlers
END [name];
```

```sql
CREATE OR REPLACE PROCEDURE raise_salary (
  p_emp_id IN employees.emp_id%TYPE,
  p_pct    IN NUMBER DEFAULT 10
) IS
BEGIN
  IF p_pct <= 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Percent must be positive');
  END IF;

  UPDATE employees
  SET    salary = salary + salary * p_pct / 100
  WHERE  emp_id = p_emp_id;

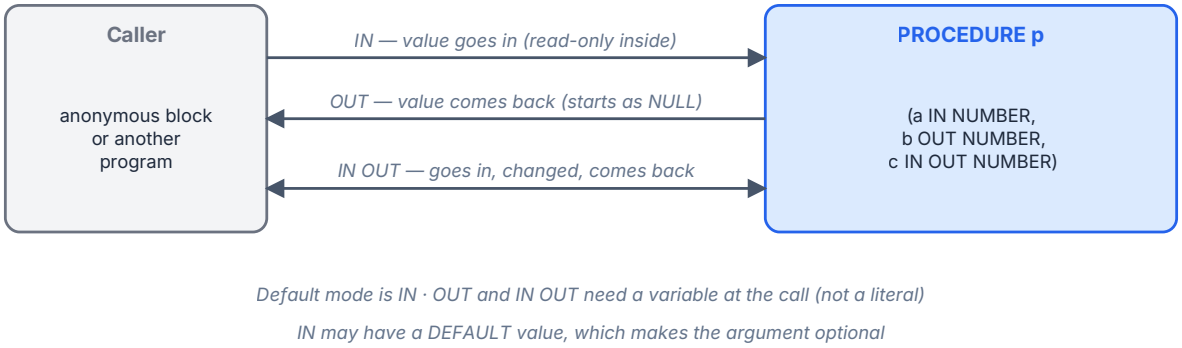
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'No such employee: ' || p_emp_id);
  END IF;
END raise_salary;
/
```

```sql
EXEC raise_salary(101, 10);                          -- positional: Amit 60000 -> 66000

BEGIN
  raise_salary(p_pct => 5, p_emp_id => 102);         -- named: Neha 45000 -> 47250
END;
/
ROLLBACK;                                            -- restore the sample data
```

If a check fails, `RAISE_APPLICATION_ERROR` stops the procedure before any damage. After `CREATE`, if SQL*Plus says *"Warning: Procedure created with compilation errors"*, run `SHOW ERRORS` to see the line and message.

Parameter modes



Default mode is IN · OUT and IN OUT need a variable at the call (not a literal)

IN may have a DEFAULT value, which makes the argument optional

Fig. 21 — IN passes a value in, OUT sends one back, IN OUT does both

Mode	Direction	Inside the subprogram	Actual argument at the call
IN	caller → subprogram	Read-only (like a constant)	Variable, literal or expression
OUT	subprogram → caller	Starts as NULL; assign it before you read it	Must be a variable
IN OUT	both	Read and write	Must be a variable

```sql
CREATE OR REPLACE PROCEDURE get_emp_info (
  p_id   IN  NUMBER,
  p_name OUT VARCHAR2,
  p_sal  OUT NUMBER
) IS
BEGIN
  SELECT emp_name, salary INTO p_name, p_sal FROM employees WHERE emp_id = p_id;
END get_emp_info;
/

CREATE OR REPLACE PROCEDURE swap (p_a IN OUT NUMBER, p_b IN OUT NUMBER) IS
  v_t NUMBER;
BEGIN
  v_t := p_a;  p_a := p_b;  p_b := v_t;
END swap;
/

DECLARE
  v_name VARCHAR2(50);
  v_sal  NUMBER;
  a NUMBER := 5;
  b NUMBER := 9;
BEGIN
  get_emp_info(103, v_name, v_sal);
  DBMS_OUTPUT.PUT_LINE(v_name || ' earns ' || v_sal);
  swap(a, b);
  DBMS_OUTPUT.PUT_LINE('a = ' || a || ', b = ' || b);
END;
/
```

```text
```

```
Ravi earns 35000
a = 9, b = 5
```

Notation	Example	Remark
Positional	`raise_salary(101, 10)`	Order must match the definition
Named	`raise_salary(p_pct => 5, p_emp_id => 102)`	Any order; readable; skips defaults
Mixed	`raise_salary(101, p_pct => 5)`	Positional arguments must come first

3.3 Stored functions

Definition

A **function** is a named PL/SQL block, stored in the database, that computes and **returns exactly one value** through a `RETURN` statement. Because it returns a value it can be used inside expressions and SQL statements.

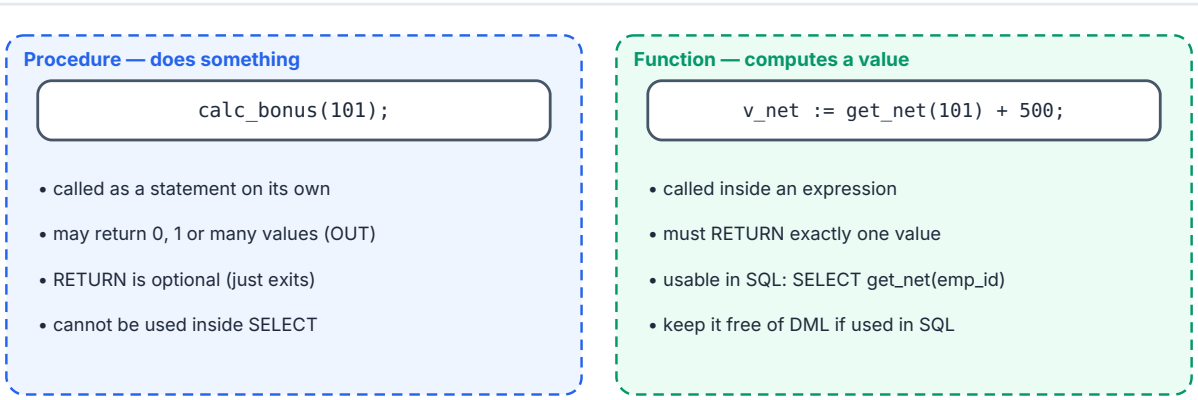


Fig. 22 — A procedure is called as a statement; a function is used inside an expression

```sql
CREATE OR REPLACE FUNCTION get_net_salary (
  p_emp_id IN employees.emp_id%TYPE
) RETURN NUMBER IS
  v_sal employees.salary%TYPE;
  v_tax NUMBER;
BEGIN
  SELECT salary INTO v_sal FROM employees WHERE emp_id = p_emp_id;
  v_tax := CASE WHEN v_sal > 50000 THEN 0.20
                WHEN v_sal > 30000 THEN 0.10
                ELSE 0 END;
  RETURN v_sal - v_sal * v_tax;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END get_net_salary;
/

SELECT emp_name, get_net_salary(emp_id) AS net_salary
FROM   employees
WHERE  dept_id = 10
ORDER  BY emp_id;
```

```text
```

```
EMP_NAME   NET_SALARY
---------- ----------
Amit            48000
Neha            40500
Karan           28000
```

A recursive function

```sql
CREATE OR REPLACE FUNCTION fact (n IN NUMBER) RETURN NUMBER IS
BEGIN
  IF n <= 1 THEN
    RETURN 1;
  END IF;
  RETURN n * fact(n - 1);
END fact;
/
SELECT fact(5) AS result FROM dual;
```

```
    RESULT
----------
       120
```

Point	Procedure	Function
Purpose	Perform an action	Compute and return a value
RETURN	Optional (just exits); no value	Mandatory, with a value of the declared type
How it is called	As a statement: `p(1);`	In an expression: `x := f(1);`
Values handed back	0, 1 or many (OUT parameters)	Exactly one (RETURN); OUT is possible but discouraged
Inside SELECT	No	Yes, if it performs no DML

⚠️ Careful

A function called from `SELECT` must not do INSERT/UPDATE/DELETE or COMMIT (ORA-14551 / ORA-14552). Make sure **every** path ends with a `RETURN`, or you get ORA-06503 "function returned without value".

🧠 Yaad rakho

Procedure = **kaam karo**. Function = **value do**. Function SELECT ke andar chal sakta hai, procedure nahi.

3.4 Packages

📘 Definition

A **package** is a schema object that groups logically related procedures, functions, variables, constants and cursors. It has two parts: a **specification** (the public interface) and a **body** (the implementation and private items).

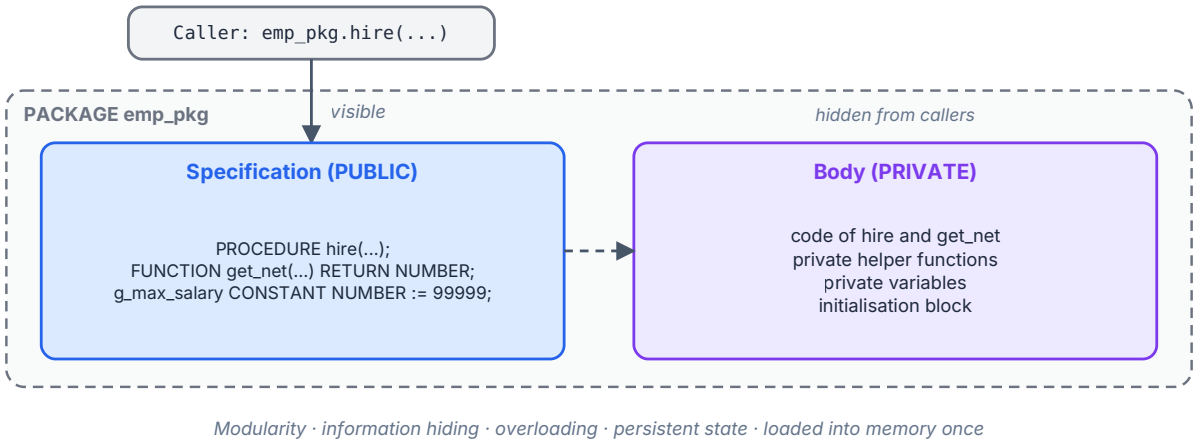


Fig. 23 — Callers see only the specification; the body stays hidden

```sql
CREATE OR REPLACE PACKAGE emp_pkg IS
  g_max_pct CONSTANT NUMBER := 25;
  PROCEDURE raise_salary (p_emp_id IN NUMBER, p_pct IN NUMBER);
  FUNCTION  dept_payroll (p_dept IN NUMBER) RETURN NUMBER;
END emp_pkg;
/

CREATE OR REPLACE PACKAGE BODY emp_pkg IS
  FUNCTION valid_pct (p_pct NUMBER) RETURN BOOLEAN IS     -- private helper
  BEGIN
    RETURN p_pct > 0 AND p_pct <= g_max_pct;
  END valid_pct;

  PROCEDURE raise_salary (p_emp_id IN NUMBER, p_pct IN NUMBER) IS
  BEGIN
    IF NOT valid_pct(p_pct) THEN
      RAISE_APPLICATION_ERROR(-20010, 'Percent must be 1 to ' || g_max_pct);
    END IF;
    UPDATE employees SET salary = salary * (1 + p_pct / 100) WHERE emp_id = p_emp_id;
  END raise_salary;

  FUNCTION dept_payroll (p_dept IN NUMBER) RETURN NUMBER IS
    v_total NUMBER;
  BEGIN
    SELECT NVL(SUM(salary), 0) INTO v_total FROM employees WHERE dept_id = p_dept;
    RETURN v_total;
  END dept_payroll;
END emp_pkg;
/
```

```sql
SELECT emp_pkg.dept_payroll(10) AS it_payroll FROM dual;
EXEC emp_pkg.raise_salary(101, 40);
```

```text
IT_PAYROLL
----------
    133000

ERROR at line 1:
ORA-20010: Percent must be 1 to 25
ORA-06512: at "YOURUSER.EMP_PKG", line 11
ORA-06512: at line 1
```

Callers use the dot notation `package.item`. They can call `emp_pkg.raise_salary` but cannot call `valid_pct` — it exists only in the body.

Benefit	Why
Modularity	Related code lives together; one place to maintain
Information hiding	Private items are invisible; change the body without breaking callers
Better performance	The whole package is loaded into memory on first call
Overloading	Several subprograms with the same name but different parameters
Persistent state	Package variables keep their value for the whole session
Fewer dependency problems	Changing only the body does not invalidate programs that call it

⭐ **Exam tip**

"Differentiate package specification and body": the spec holds **declarations** (public, what callers may use); the body holds the **code** and private items. The spec can exist without a body only if it declares no subprograms.

3.5 Managing stored code

Task	Command
See compile errors	`SHOW ERRORS` or `SELECT * FROM user_errors WHERE name = 'RAISE_SALARY';`
Read the source text	`SELECT text FROM user_source WHERE name = 'EMP_PKG' ORDER BY line;`
List your program units and status	`SELECT object_name, object_type, status FROM user_objects;`
Recompile	`ALTER PROCEDURE raise_salary COMPILE;`
Allow another user to run it	`GRANT EXECUTE ON raise_salary TO app_user;`
Remove it	`DROP PROCEDURE raise_salary;`

The status is **VALID** when the code compiled and its dependencies are intact, and **INVALID** when a table or program it uses has changed — Oracle recompiles it automatically on the next call if it can.

📖 GFG: PL/SQL exception handling · Procedures and functions · Packages

❓ **Likely exam questions — Unit III**

- What is an exception? Explain predefined, non-predefined and user-defined exceptions with examples.
- Explain `RAISE_APPLICATION_ERROR`, SQLCODE and SQLERRM.
- How do exceptions propagate through nested blocks?
- Differentiate between a procedure and a function.
- Explain the IN, OUT and IN OUT parameter modes with an example.
- Write a function that returns the net salary of an employee, and call it from SELECT.
- What is a package? Explain specification, body and the advantages of packages.

UNIT IV

Triggers, Collections and Bulk Processing

Trigger types | :OLD / :NEW | Mutating table | Records | Collections | BULK COLLECT | FORALL | Autonomous transaction

4.1 Triggers

Definition

A **trigger** is a named PL/SQL block stored in the database that Oracle **runs automatically** when a specified event occurs — a DML statement on a table, a DDL statement, or a database event such as LOGON. You never call a trigger; the event fires it.

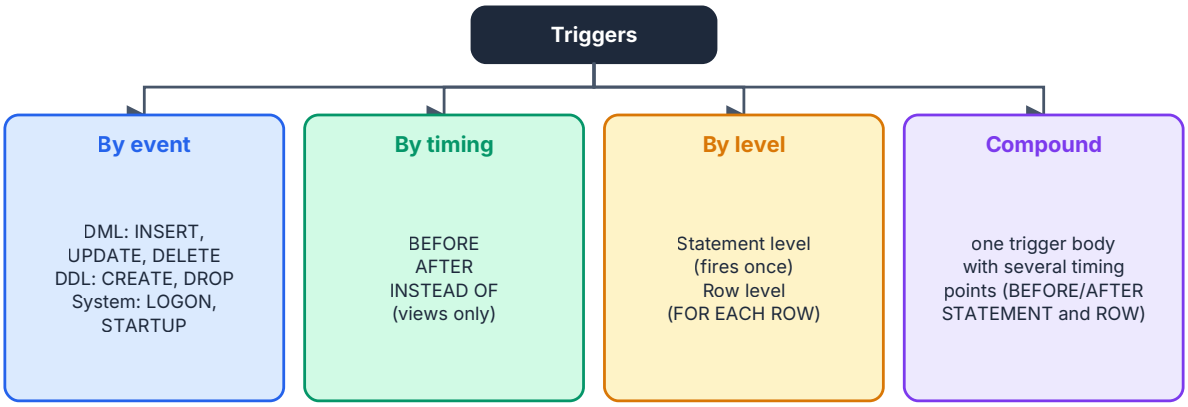


Fig. 24 — Triggers can be classified by event, timing, level and form

```sql
CREATE [OR REPLACE] TRIGGER trigger_name
{ BEFORE | AFTER | INSTEAD OF }
{ INSERT | UPDATE [OF column] | DELETE } [OR ...]
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
DECLARE
  -- optional declarations
BEGIN
  -- trigger body
END;
```

Firing order

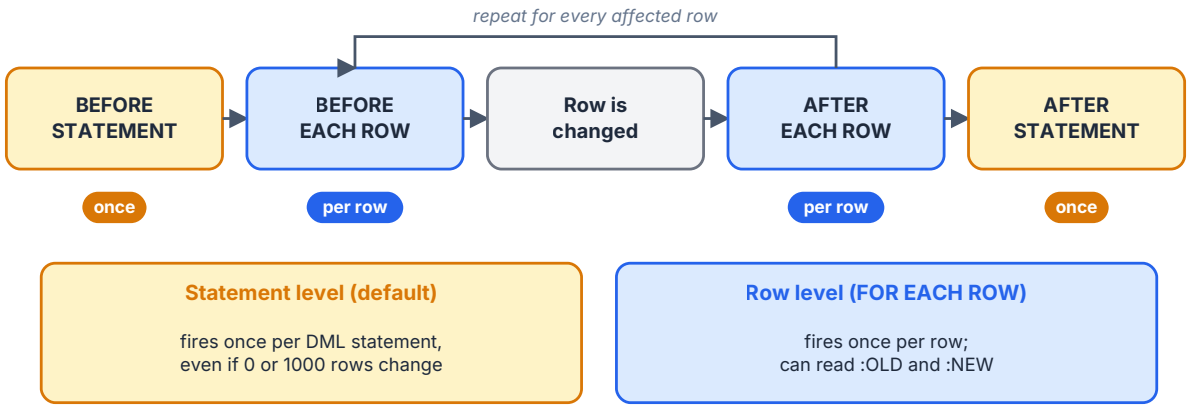


Fig. 25 — The five firing points of one DML statement

:OLD and :NEW

Inside a **row-level** trigger, `:OLD` holds the row as it was and `:NEW` holds the row as it will be.

Event	:OLD (row before)	:NEW (row after)
INSERT	NULL in every field	the new values
UPDATE	values before the change	values after the change
DELETE	values being deleted	NULL in every field

In a BEFORE row trigger you may assign :NEW.col := ... to change the row being saved

In SQL inside the trigger write :OLD.salary — in the WHEN clause write OLD.salary (no colon)

Fig. 26 — Which pseudo-records are filled for each event

Example 1 — a BEFORE row trigger that cleans data

```sql
CREATE OR REPLACE TRIGGER trg_emp_before_ins
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
  :NEW.emp_name := INITCAP(:NEW.emp_name);          -- 'pooja' becomes 'Pooja'
  IF :NEW.hire_date IS NULL THEN
    :NEW.hire_date := SYSDATE;
  END IF;
END;
/

INSERT INTO employees (emp_id, emp_name, dept_id, salary)
VALUES (106, 'pooja', 30, 42000);

SELECT emp_id, emp_name, hire_date FROM employees WHERE emp_id = 106;
ROLLBACK;
```

```text
```

```
    EMP_ID EMP_NAME   HIRE_DATE
---------- ---------- ---------
       106 Pooja      (today's date)
```

Example 2 — an AFTER row trigger that writes an audit trail

```sql
CREATE TABLE salary_audit (
  audit_id    NUMBER GENERATED ALWAYS AS IDENTITY,
  emp_id      NUMBER,
  old_salary  NUMBER,
  new_salary  NUMBER,
  changed_by  VARCHAR2(30),
  changed_on  DATE
);

CREATE OR REPLACE TRIGGER trg_salary_audit
AFTER UPDATE OF salary ON employees
FOR EACH ROW
WHEN (OLD.salary <> NEW.salary)
BEGIN
  INSERT INTO salary_audit (emp_id, old_salary, new_salary, changed_by, changed_on)
  VALUES (:OLD.emp_id, :OLD.salary, :NEW.salary, USER, SYSDATE);
END;
/

UPDATE employees SET salary = 66000 WHERE emp_id = 101;
SELECT emp_id, old_salary, new_salary FROM salary_audit;
```

```
    EMP_ID OLD_SALARY NEW_SALARY
---------- ---------- ----------
       101      60000      66000
```

The `WHEN` clause is evaluated before the body runs, so updates that leave the salary unchanged do not create audit rows. Note that inside `WHEN` you write `OLD` and `NEW` **without** the colon.

Example 3 — a BEFORE trigger that enforces a rule

```sql
CREATE OR REPLACE TRIGGER trg_no_salary_cut
BEFORE UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
  IF :NEW.salary < :OLD.salary THEN
    RAISE_APPLICATION_ERROR(-20020, 'Salary cannot be reduced');
  END IF;
END;
/

UPDATE employees SET salary = 30000 WHERE emp_id = 101;
```

```
ERROR at line 1:
ORA-20020: Salary cannot be reduced
(followed by an ORA-06512 line that names the trigger)
```

The error also rolls back the failed statement, so the row keeps its old salary.

Use a BEFORE trigger to	Use an AFTER trigger to
Validate data, reject bad rows, or change `:NEW` values before they are saved	Write audit rows, update summary tables, notify other systems — work that depends on the change having happened

INSTEAD OF triggers (on views)

A complex view (a join, for example) is not directly updatable. An **INSTEAD OF** trigger intercepts the DML on the view and performs the real changes on the base tables.

```sql
CREATE OR REPLACE VIEW v_emp_dept AS
  SELECT e.emp_id, e.emp_name, d.dept_name
  FROM   employees e JOIN departments d ON d.dept_id = e.dept_id;

CREATE OR REPLACE TRIGGER trg_v_emp_dept_ins
INSTEAD OF INSERT ON v_emp_dept
FOR EACH ROW
DECLARE
  v_dept departments.dept_id%TYPE;
BEGIN
  SELECT dept_id INTO v_dept FROM departments WHERE dept_name = :NEW.dept_name;
  INSERT INTO employees (emp_id, emp_name, dept_id) VALUES (:NEW.emp_id, :NEW.emp_name, v_dept);
END;
/
```

The mutating-table error

> ⚠️ **Careful — ORA-04091**
>
> A **row-level** trigger must not read or modify the very table that is being changed by the statement that fired it — Oracle raises `ORA-04091: table is mutating`. Typical cure: move the logic to an **AFTER STATEMENT** part of a **compound trigger**, or redesign so the trigger only touches other tables.

Task	Command
Disable / enable one trigger	`ALTER TRIGGER trg_salary_audit DISABLE;` / `ENABLE;`
Disable all triggers of a table	`ALTER TABLE employees DISABLE ALL TRIGGERS;`
List triggers	`SELECT trigger_name, trigger_type, status FROM user_triggers;`
Remove	`DROP TRIGGER trg_salary_audit;`

> 🏭 **Real-life use case — Real-life triggers**
>
> Audit trails (who changed what), auto-filling created / updated timestamps, enforcing rules that need more than a CHECK constraint, keeping a denormalised summary table in step, and blocking DDL on production schemas.

⭐ **Exam tip**

Examiners like the comparison **trigger vs procedure**: a trigger fires **automatically** on an event, takes **no parameters** and cannot contain COMMIT or ROLLBACK; a procedure is **called explicitly**, takes parameters, and can control transactions.

🧠 **Yaad rakho**

Trigger = **apne aap chalta hai**. Row trigger me **:OLD / :NEW**. BEFORE = check/badlo, AFTER = record/audit. Same table padhoge to **mutating** error.

4.2 Records and collections

User-defined records

📘 **Definition**

A **record** groups related fields of different types into one variable, like a C struct. `%ROWTYPE` builds one from a table; `TYPE ... IS RECORD` builds a custom one.

```sql
DECLARE
  TYPE t_emp IS RECORD (
    name employees.emp_name%TYPE,
    sal  employees.salary%TYPE
  );
  r t_emp;
BEGIN
  SELECT emp_name, salary INTO r FROM employees WHERE emp_id = 101;
  DBMS_OUTPUT.PUT_LINE(r.name || ' - ' || r.sal);
END;
/
```

```text
Amit - 60000
```

Collections

📘 **Definition**

A **collection** is an ordered group of elements of the same type, accessed by an index — PL/SQL's version of an array or list. There are three kinds: **associative array**, **nested table** and **VARRAY**.

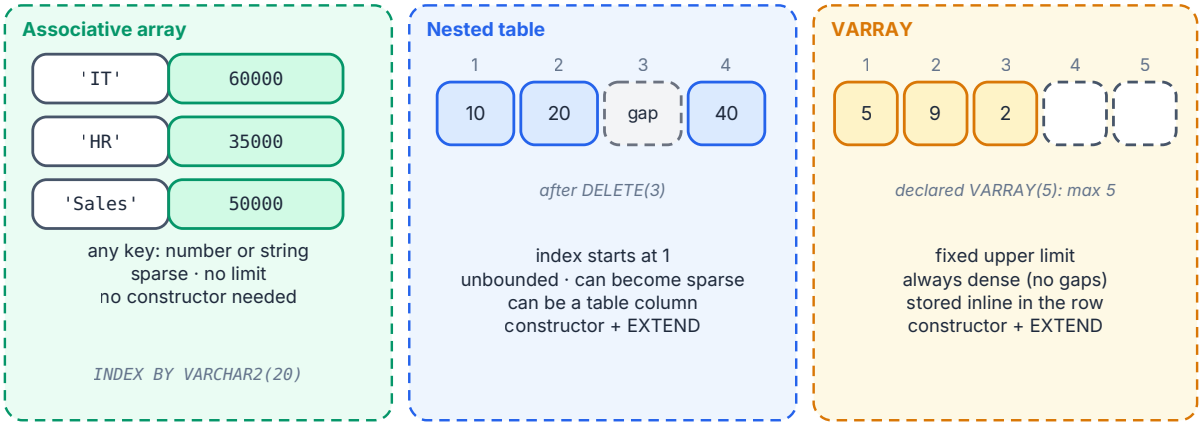


Fig. 27 — Associative array, nested table and VARRAY side by side

Point	Associative array	Nested table	VARRAY
Index	Number or string	Integer from 1	Integer from 1
Upper limit	None	None	Fixed at declaration
Sparse (gaps)	Yes	Can become sparse (DELETE)	No — always dense
Needs constructor	No	Yes	Yes
Can be a table column	No	Yes	Yes
Declared with	`TABLE OF t INDEX BY k`	`TABLE OF t`	`VARRAY(n) OF t`

```sql
-- Associative array: keyed by department name
DECLARE
  TYPE t_pay IS TABLE OF NUMBER INDEX BY VARCHAR2(20);
  v_pay t_pay;
  v_key VARCHAR2(20);
BEGIN
  v_pay('IT')    := 60000;
  v_pay('HR')    := 35000;
  v_pay('Sales') := 50000;
  v_key := v_pay.FIRST;
  WHILE v_key IS NOT NULL LOOP
    DBMS_OUTPUT.PUT_LINE(v_key || ' -> ' || v_pay(v_key));
    v_key := v_pay.NEXT(v_key);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Count = ' || v_pay.COUNT);
END;
/
```

```text
HR -> 35000
IT -> 60000
Sales -> 50000
Count = 3
```

```sql
-- Nested table: constructor, EXTEND, DELETE leaves a gap
DECLARE
  TYPE t_names IS TABLE OF VARCHAR2(30);
  v_names t_names := t_names('Amit', 'Neha', 'Ravi');
```

```
BEGIN
  v_names.EXTEND;                -- add one empty slot at position 4
  v_names(4) := 'Sara';
  v_names.DELETE(2);             -- position 2 becomes a gap
  FOR i IN v_names.FIRST .. v_names.LAST LOOP
    IF v_names.EXISTS(i) THEN
      DBMS_OUTPUT.PUT_LINE(i || ': ' || v_names(i));
    END IF;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('COUNT = ' || v_names.COUNT || ', LAST = ' || v_names.LAST);
END;
/
```

```
TEXT
1: Amit
3: Ravi
4: Sara
COUNT = 3, LAST = 4
```

```
SQL
-- VARRAY: bounded
DECLARE
  TYPE t_marks IS VARRAY(5) OF NUMBER;
  v_marks t_marks := t_marks(70, 85, 90);
BEGIN
  v_marks.EXTEND;
  v_marks(4) := 60;
  DBMS_OUTPUT.PUT_LINE('Count = ' || v_marks.COUNT || ', Limit = ' || v_marks.LIMIT);
END;
/
```

```
TEXT
Count = 4, Limit = 5
```

Method	Returns / does
`COUNT`	Number of elements that exist
`FIRST`, `LAST`	Smallest and largest index in use
`NEXT(n)`, `PRIOR(n)`	Next / previous index after n (NULL if none)
`EXISTS(n)`	TRUE if element n exists
`EXTEND[(k)]`	Add k empty elements (nested table, VARRAY)
`TRIM[(k)]`	Remove k elements from the end
`DELETE[(n)]`	Remove element n, or all elements
`LIMIT`	Maximum size of a VARRAY

⚠ Careful

Common collection errors: `COLLECTION_IS_NULL` (using a nested table never initialised with its constructor), `SUBSCRIPT_BEYOND_COUNT` (index above COUNT without EXTEND), `SUBSCRIPT_OUTSIDE_LIMIT` (index above a VARRAY limit) and `NO_DATA_FOUND` (reading a deleted or never-assigned element).

🧠 **Yaad rakho**

Teen collections: **Associative** (key-value, koi limit nahi), **Nested table** (1 se, gap aa sakte hain), **VARRAY** (fixed limit, gap nahi).

4.3 BULK COLLECT and FORALL

📘 **Definition**

BULK COLLECT fetches many rows into collections with **one** trip to the SQL engine. **FORALL** sends a whole collection of values to one DML statement with **one** trip. Both cut the number of context switches.

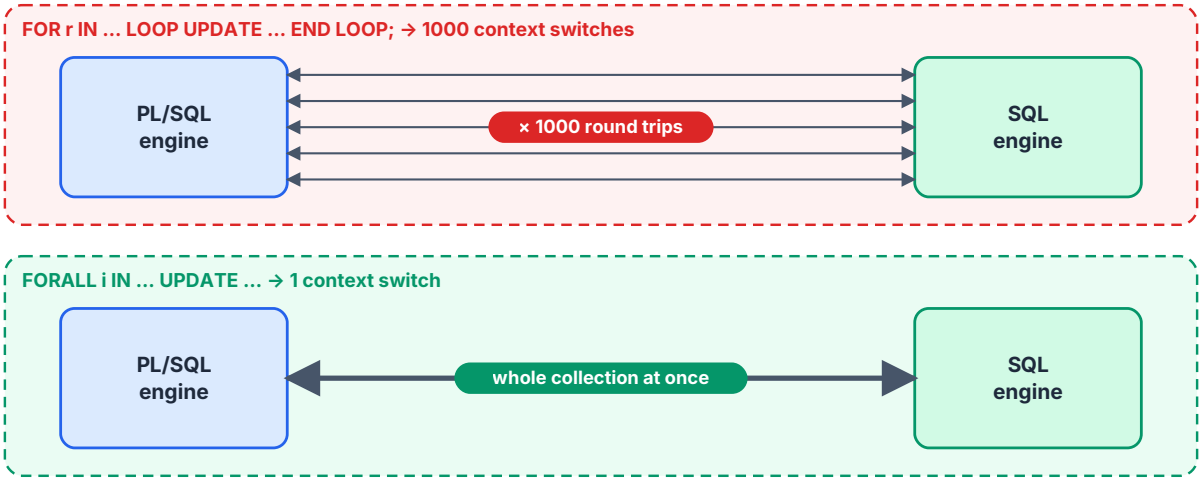


Fig. 28 — A row-by-row loop switches engines for every row; FORALL switches once

```sql
DECLARE
  TYPE t_names IS TABLE OF employees.emp_name%TYPE;
  TYPE t_sals  IS TABLE OF employees.salary%TYPE;
  v_names t_names;
  v_sals  t_sals;
BEGIN
  SELECT emp_name, salary BULK COLLECT INTO v_names, v_sals
  FROM   employees
  WHERE  dept_id = 10
  ORDER  BY emp_id;

  FOR i IN 1 .. v_names.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(v_names(i) || '  ' || v_sals(i));
  END LOOP;
END;
/
```

```text
Amit  60000
Neha  45000
Karan  28000
```

```sql
DECLARE
  TYPE t_ids IS TABLE OF employees.emp_id%TYPE;
  v_ids t_ids := t_ids(101, 102, 105);
BEGIN
```

```
  FORALL i IN 1 .. v_ids.COUNT
    UPDATE employees SET salary = salary * 1.05 WHERE emp_id = v_ids(i);
  DBMS_OUTPUT.PUT_LINE('Rows updated: ' || SQL%ROWCOUNT);
  ROLLBACK;
END;
/
```

```
TEXT
Rows updated: 3
```

Large tables: fetch in batches with LIMIT

A BULK COLLECT without a limit loads every row into memory (PGA). For big tables, fetch a fixed batch at a time.

```
SQL
DECLARE
  CURSOR c IS SELECT emp_id, salary FROM employees;
  TYPE t_ids  IS TABLE OF employees.emp_id%TYPE;
  TYPE t_sals IS TABLE OF employees.salary%TYPE;
  v_ids  t_ids;
  v_sals t_sals;
BEGIN
  OPEN c;
  LOOP
    FETCH c BULK COLLECT INTO v_ids, v_sals LIMIT 1000;
    EXIT WHEN v_ids.COUNT = 0;                    -- not c%NOTFOUND (see warning)
    FORALL i IN 1 .. v_ids.COUNT
      UPDATE employees SET salary = v_sals(i) * 1.02 WHERE emp_id = v_ids(i);
  END LOOP;
  CLOSE c;
  ROLLBACK;
END;
/
```

> ⚠️ **Careful**
>
> With `LIMIT`, `c%NOTFOUND` becomes TRUE as soon as the **last, partly filled** batch is fetched — exiting on it would skip those rows. Test `COLLECTION.COUNT = 0` after the fetch instead.

When some rows fail: SAVE EXCEPTIONS

By default one failing row aborts the whole FORALL. `SAVE EXCEPTIONS` keeps going and collects the failures in `SQL%BULK_EXCEPTIONS`.

```
SQL
BEGIN
  FORALL i IN 1 .. v_ids.COUNT SAVE EXCEPTIONS
    INSERT INTO employees (emp_id, emp_name) VALUES (v_ids(i), v_names(i));
EXCEPTION
  WHEN OTHERS THEN
    IF SQLCODE = -24381 THEN                      -- ORA-24381: errors in array DML
      FOR j IN 1 .. SQL%BULK_EXCEPTIONS.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('Row ' || SQL%BULK_EXCEPTIONS(j).ERROR_INDEX || ': '
          || SQLERRM(-SQL%BULK_EXCEPTIONS(j).ERROR_CODE));
      END LOOP;
    ELSE
      RAISE;
    END IF;
END;
```

💡 **Why it matters**

Each switch between the PL/SQL and SQL engines has a small fixed cost. Multiplied by tens of thousands of rows it dominates the run time. Bulk operations are the single biggest tuning step for batch PL/SQL; measure on your own data to see the gain.

🧠 **Yaad rakho**

BULK COLLECT = bahut rows **ek saath padho**. **FORALL** = bahut rows **ek saath likho**. Bade table pe **LIMIT** zaroor.

4.4 Autonomous transactions

📘 **Definition**

PRAGMA AUTONOMOUS_TRANSACTION makes a subprogram run in its own, independent transaction: it can COMMIT or ROLLBACK without affecting the caller's transaction.

The classic use is an error logger that must save its row even when the main transaction is rolled back.

```sql
CREATE TABLE error_log (
  logged_on DATE,
  message   VARCHAR2(4000)
);

CREATE OR REPLACE PROCEDURE log_error (p_msg IN VARCHAR2) IS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO error_log (logged_on, message) VALUES (SYSDATE, p_msg);
  COMMIT;                          -- commits ONLY the log row
END log_error;
/
```

⚠️ **Careful**

An autonomous procedure cannot see the caller's uncommitted changes, and it **must** end with COMMIT or ROLLBACK before returning (otherwise ORA-06519). Use it for logging and auditing only — not to split real business transactions.

📖 GFG: PL/SQL triggers · PL/SQL collections · BULK COLLECT and FORALL

❓ **Likely exam questions — Unit IV**

- What is a trigger? Explain its types and the firing order with a diagram.
- What are `:OLD` and `:NEW`? Write a row trigger that audits salary changes.
- Differentiate a trigger from a stored procedure.
- What is the mutating-table error and how is it avoided?
- Explain the three collection types and compare them.
- List the collection methods with their use.
- Explain BULK COLLECT and FORALL. Why are they faster?
- What is an autonomous transaction? Give one use.

UNIT V

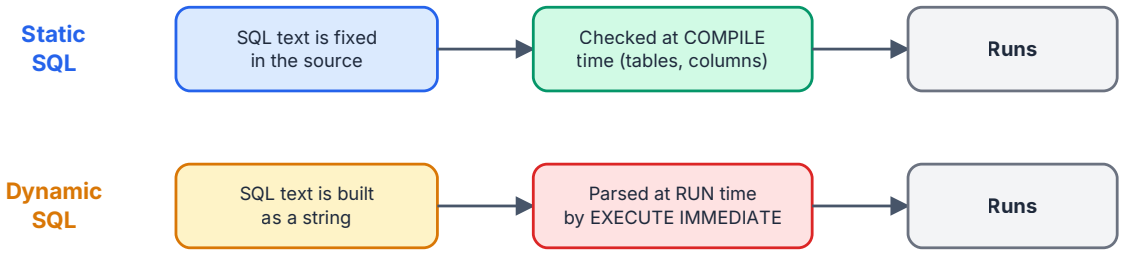
Dynamic SQL, Tuning and a Mini Project

EXECUTE IMMEDIATE | SQL injection | Invoker rights | Performance | Naming standards | Bank-transfer project | Practice programs

5.1 Dynamic SQL

Definition

Dynamic SQL is a SQL statement that is built as a **text string at run time** and executed with `EXECUTE IMMEDIATE` (or `OPEN ... FOR` for multi-row queries). **Static SQL** is written in the source and checked when the program is compiled.



Use dynamic SQL for: DDL inside PL/SQL · table or column names chosen at run time · flexible WHERE clauses

The price: errors show up only when it runs, and values must be passed as binds

Fig. 29 — Static SQL is checked at compile time; dynamic SQL is parsed when it runs

Point	Static SQL	Dynamic SQL
When parsed	At compile time	At run time
Errors found	While compiling	Only when executed
Allowed statements	DML, SELECT INTO	Anything, including DDL
Table / column names	Fixed	Can be decided at run time
Speed and safety	Faster, dependency-tracked	Slower, harder to test

```sql
-- 1. DDL inside PL/SQL
BEGIN
  EXECUTE IMMEDIATE 'CREATE TABLE temp_scores (id NUMBER, score NUMBER)';
END;
/
```

```
-- 2. DML with bind variables (:amt, :d) supplied by USING
DECLARE
  v_sql VARCHAR2(200) := 'UPDATE employees SET salary = salary + :amt WHERE dept_id = :d';
BEGIN
  EXECUTE IMMEDIATE v_sql USING 1000, 20;
  DBMS_OUTPUT.PUT_LINE('Rows updated: ' || SQL%ROWCOUNT);
  ROLLBACK;
END;
/

-- 3. Query with INTO; the table name is chosen at run time
DECLARE
  v_table VARCHAR2(30) := 'EMPLOYEES';
  v_count NUMBER;
BEGIN
  EXECUTE IMMEDIATE 'SELECT COUNT(*) FROM ' || DBMS_ASSERT.SIMPLE_SQL_NAME(v_table)
    INTO v_count;
  DBMS_OUTPUT.PUT_LINE(v_table || ' has ' || v_count || ' rows');
END;
/
```

```
TEXT
Rows updated: 1
EMPLOYEES has 5 rows
```

Only Ravi (department 20) is updated in example 2. A **bind variable** can stand in for a *value* but never for a table or column name — those must be concatenated into the string, so validate them (`DBMS_ASSERT`).

SQL injection — the one danger

```
SQL
-- DANGEROUS: user input pasted into the SQL text
EXECUTE IMMEDIATE 'DELETE FROM employees WHERE emp_name = ''' || p_name || '''';
-- p_name = x' OR 'a'='a    deletes every row!

-- SAFE: user input travels as a bind value
EXECUTE IMMEDIATE 'DELETE FROM employees WHERE emp_name = :n' USING p_name;
```

> ⚠️ **Careful**
>
> Never build a statement by joining raw user input into the text. Use **bind variables** for values and **DBMS_ASSERT** for names. Bind variables are also faster, because Oracle can reuse one parsed statement for every value.

> 🧠 **Yaad rakho**
>
> Static = **compile time** pe check. Dynamic = **run time** pe. Value ke liye **:bind + USING**, naam ke liye **DBMS_ASSERT**. Kabhi user input seedha jodo mat.

5.2 Definer rights and invoker rights

> 📘 **Definition**
>
> A stored program normally runs with the privileges of its **owner** (**definer rights**, the default). Adding `AUTHID CURRENT_USER` makes it run with the privileges — and the schema objects — of whoever **calls** it (**invoker rights**).

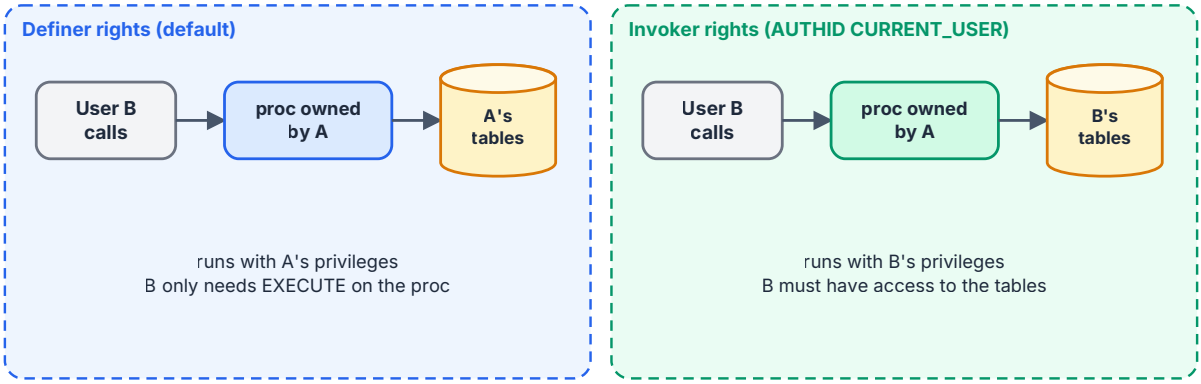


Fig. 30 — Definer rights use the owner's privileges; invoker rights use the caller's

```sql
CREATE OR REPLACE PROCEDURE show_count
  AUTHID CURRENT_USER           -- invoker rights; omit for definer rights
IS
  v_n NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_n FROM employees;    -- whose EMPLOYEES table? the caller's
  DBMS_OUTPUT.PUT_LINE('Rows: ' || v_n);
END;
/
```

Use	When
Definer rights	Users should reach data **only through the procedure** and have no direct table access — the usual security pattern
Invoker rights	The same utility code must work on each user's own tables (for example in multi-schema applications)

5.3 Performance and best practices



Fig. 31 — Six habits that make PL/SQL faster

Practice	Why / how
Prefer set-based SQL	One `UPDATE ... WHERE` usually beats a cursor loop that updates row by row
Bulk operations	`BULK COLLECT` + `FORALL` with `LIMIT` (Unit IV)
Bind variables	Fewer parses, no SQL injection
Do not commit in a loop without need	Frequent commits slow the job and can break atomicity (and FOR UPDATE cursors)
Use `PLS_INTEGER` for counters	Native integer arithmetic, faster than NUMBER
Use `%TYPE` / `%ROWTYPE`	Code survives column changes
Pass big collections with `NOCOPY`	OUT / IN OUT parameters are copied by default; NOCOPY passes a reference
Cache repeated look-ups	Declare a function with `RESULT_CACHE`, or `DETERMINISTIC` if it always gives the same output for the same input
Select only what you need	List columns instead of `SELECT *` in production code
Turn on compiler warnings	`ALTER SESSION SET PLSQL_WARNINGS = 'ENABLE:ALL';`

Naming conventions

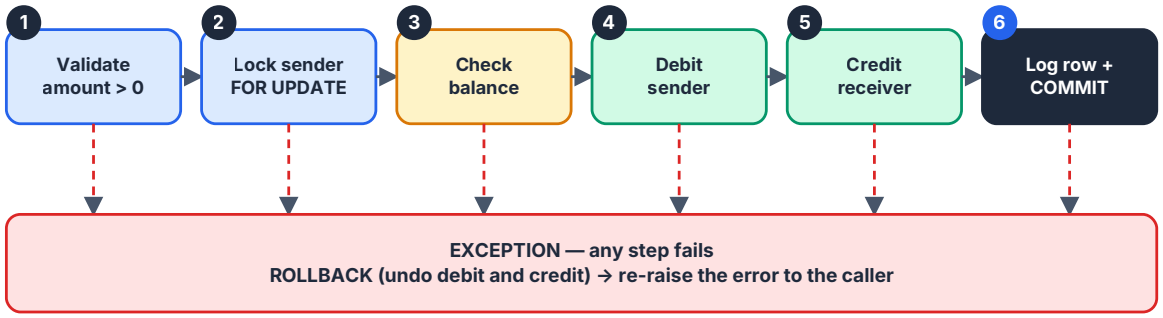
Prefix	Used for	Example
`v_`	Local variable	`v_total`
`p_`	Parameter	`p_emp_id`
`c_`	Constant or cursor	`c_gst`, `c_emp`
`e_`	User-defined exception	`e_low_salary`
`g_`	Package (global) variable	`g_max_pct`
`t_`	User-defined type	`t_names`
`trg_`	Trigger	`trg_salary_audit`

⭐ **Exam tip**

For a "best practices" question write six points: **bind variables, bulk operations, %TYPE, specific exception handling, short transactions, packages**. Add one example for each and you will have a full-marks answer.

5.4 Mini project — bank money transfer

This one procedure uses almost everything in the book: parameters, validation with user errors, row locking, DML, exception handling and transaction control.



All-or-nothing (atomicity): money is never debited without being credited

Fig. 32 — Six steps; any failure rolls everything back

```sql
CREATE TABLE accounts (
  acc_id   NUMBER PRIMARY KEY,
  holder   VARCHAR2(50),
  balance  NUMBER(12,2) CHECK (balance >= 0)
);

CREATE TABLE txn_log (
  txn_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  from_acc  NUMBER,
  to_acc    NUMBER,
  amount    NUMBER(12,2),
  txn_time  TIMESTAMP DEFAULT SYSTIMESTAMP
);

INSERT INTO accounts VALUES (1, 'Amit', 50000);
INSERT INTO accounts VALUES (2, 'Neha', 20000);
COMMIT;
```

```sql
CREATE OR REPLACE PROCEDURE transfer_funds (
  p_from   IN accounts.acc_id%TYPE,
  p_to     IN accounts.acc_id%TYPE,
  p_amount IN NUMBER
) IS
  v_balance accounts.balance%TYPE;
BEGIN
  IF p_amount <= 0 THEN
    RAISE_APPLICATION_ERROR(-20101, 'Amount must be positive');
  END IF;
  IF p_from = p_to THEN
    RAISE_APPLICATION_ERROR(-20102, 'Sender and receiver must differ');
  END IF;

  SELECT balance INTO v_balance FROM accounts
  WHERE  acc_id = p_from FOR UPDATE;             -- lock the sender's row

  IF v_balance < p_amount THEN
    RAISE_APPLICATION_ERROR(-20103, 'Insufficient balance');
  END IF;

  UPDATE accounts SET balance = balance - p_amount WHERE acc_id = p_from;
  UPDATE accounts SET balance = balance + p_amount WHERE acc_id = p_to;
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20104, 'Receiver account not found');
  END IF;
```

```
  INSERT INTO txn_log (from_acc, to_acc, amount) VALUES (p_from, p_to, p_amount);
  COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN                        -- sender row missing
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20105, 'Sender account not found');
  WHEN OTHERS THEN
    ROLLBACK;                                    -- undo debit and credit
    RAISE;                                       -- tell the caller what went wrong
END transfer_funds;
/
```

SQL

```
EXEC transfer_funds(1, 2, 15000);
SELECT acc_id, holder, balance FROM accounts ORDER BY acc_id;

EXEC transfer_funds(1, 2, 99999);
```

TEXT

```
    ACC_ID HOLDER        BALANCE
---------- ---------- ----------
         1 Amit            35000
         2 Neha            35000

ERROR at line 1:
ORA-20103: Insufficient balance
```

After the failed second call the balances are unchanged — the transfer is **atomic**. The `FOR UPDATE` lock stops two simultaneous transfers from reading the same balance and overdrawing the account.

💡 **Why it matters**

Here the procedure owns the whole business transaction, so it commits. In a layered application you would often leave the COMMIT to the caller (so several procedures can join one transaction) and keep only the ROLLBACK-and-re-raise logic.

🏭 **Real-life use case — Extend it**

Add a `trg_accounts_audit` trigger for balance changes, wrap the procedure in a package `bank_pkg`, expose a `get_statement` function that returns a SYS_REFCURSOR, and call it from your Java or Flutter backend.

5.5 Classic practice programs

These are the usual lab and viva programs. Type each one and change the input to test it.

SQL

```
-- Prime number check
DECLARE
  v_n     NUMBER  := 29;
  v_i     NUMBER  := 2;
  v_prime BOOLEAN := TRUE;
BEGIN
  IF v_n < 2 THEN v_prime := FALSE; END IF;
  WHILE v_prime AND v_i * v_i <= v_n LOOP
    IF MOD(v_n, v_i) = 0 THEN
      v_prime := FALSE;
    END IF;
    v_i := v_i + 1;
  END LOOP;
```

```
  IF v_prime THEN
    DBMS_OUTPUT.PUT_LINE(v_n || ' is prime');
  ELSE
    DBMS_OUTPUT.PUT_LINE(v_n || ' is not prime');
  END IF;
END;
/
```

```
TEXT
29 is prime
```

```
SQL
-- First 8 Fibonacci numbers
DECLARE
  a NUMBER := 0;
  b NUMBER := 1;
  c NUMBER;
  v_out VARCHAR2(100);
BEGIN
  FOR i IN 1 .. 8 LOOP
    v_out := v_out || a || ' ';
    c := a + b;  a := b;  b := c;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(RTRIM(v_out));
END;
/
```

```
TEXT
0 1 1 2 3 5 8 13
```

```
SQL
-- Reverse a string and test for palindrome
DECLARE
  v_str VARCHAR2(30) := 'MADAM';
  v_rev VARCHAR2(30);
BEGIN
  FOR i IN REVERSE 1 .. LENGTH(v_str) LOOP
    v_rev := v_rev || SUBSTR(v_str, i, 1);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Reverse = ' || v_rev);
  IF v_str = v_rev THEN
    DBMS_OUTPUT.PUT_LINE(v_str || ' is a palindrome');
  ELSE
    DBMS_OUTPUT.PUT_LINE(v_str || ' is not a palindrome');
  END IF;
END;
/
```

```
TEXT
Reverse = MADAM
MADAM is a palindrome
```

```
SQL
-- Highest-paid employee of every department (nested cursor FOR loops)
BEGIN
  FOR d IN (SELECT dept_id, dept_name FROM departments ORDER BY dept_id) LOOP
    FOR e IN (SELECT emp_name, salary FROM employees
              WHERE  dept_id = d.dept_id
              ORDER  BY salary DESC
              FETCH FIRST 1 ROW ONLY) LOOP
      DBMS_OUTPUT.PUT_LINE(d.dept_name || ': ' || e.emp_name || ' (' || e.salary || ')');
    END LOOP;
```

```
  END LOOP;
END;
/
```

```
TEXT
IT: Amit (60000)
HR: Ravi (35000)
Sales: Sara (50000)
```

More to try

- [ ] Armstrong number and sum of digits of a number
- [ ] Swap two numbers with and without a third variable
- [ ] Grade calculator from marks using CASE
- [ ] Function: simple interest, and a function that returns TRUE for a leap year
- [ ] Cursor report: department name, employee count and total salary (use a join)
- [ ] Procedure that gives a raise by designation and handles NO_DATA_FOUND
- [ ] Trigger that stops an employee being inserted with a future hire date
- [ ] Package `bank_pkg` with deposit, withdraw and balance, using a user-defined exception
- [ ] BULK COLLECT all employees and print those above the average salary

5.6 Quick viva answers

Question	One-line answer
Which section of a block is mandatory?	BEGIN ... END (the executable section)
Difference between `:=` and `=`?	Assignment versus comparison
What does SELECT INTO need?	Exactly one row, else NO_DATA_FOUND / TOO_MANY_ROWS
Can you use DDL directly in PL/SQL?	No — use EXECUTE IMMEDIATE
Name the cursor attributes.	%FOUND, %NOTFOUND, %ISOPEN, %ROWCOUNT
Why prefer a cursor FOR loop?	It opens, fetches, exits and closes automatically
Valid range for RAISE_APPLICATION_ERROR?	-20000 to -20999
What are :OLD and :NEW?	Row values before and after the change in a row trigger
Can a trigger COMMIT?	Not normally — only inside an autonomous transaction
Which collection has a size limit?	VARRAY
Why is FORALL fast?	One context switch for the whole collection
How to make a procedure run with the caller's rights?	AUTHID CURRENT_USER

📖 GFG: Dynamic SQL in PL/SQL · PL/SQL programs

❓ Likely exam questions — Unit V

- What is dynamic SQL? Compare it with static SQL and explain `EXECUTE IMMEDIATE` with a bind variable.
- What is SQL injection and how is it prevented in PL/SQL?
- Differentiate definer rights and invoker rights.
- List six best practices for writing efficient PL/SQL.
- Write a procedure to transfer money between two accounts with proper exception handling.
- Write PL/SQL programs for prime check, Fibonacci series and palindrome.

Quick comparison sheet

One-glance tables for the last day before the exam. Each row is a favourite "differentiate between" or "list" question.

Units I–II

Topic	Remember
SQL / PL/SQL	One statement, declarative / whole block, procedural + SQL, stored and reusable
Block parts	DECLARE (opt) · BEGIN (mandatory) · EXCEPTION (opt) · END;
Assignment / comparison	`:=` / `=`
%TYPE / %ROWTYPE	One column's type / a whole row
IF chain	IF – ELSIF – ELSE – END IF (spelling: ELSIF)
Loops	LOOP (EXIT inside) · WHILE (test first) · FOR (auto counter, REVERSE)
NULL	Use IS NULL; NVL / COALESCE to replace; NULL = NULL is not TRUE
SELECT INTO	Exactly one row; 0 → NO_DATA_FOUND; 2+ → TOO_MANY_ROWS
Implicit / explicit cursor	Oracle's own for DML and SELECT INTO / yours for multi-row queries
Explicit cursor steps	DECLARE → OPEN → FETCH (loop) → CLOSE
Cursor attributes	%FOUND · %NOTFOUND · %ISOPEN · %ROWCOUNT
Cursor FOR loop	Opens, fetches, exits and closes by itself
FOR UPDATE / CURRENT OF	Lock the rows / change the row the cursor is on
REF CURSOR	Cursor variable; SYS_REFCURSOR is the ready-made weak type
Transaction control	COMMIT permanent · ROLLBACK undo · SAVEPOINT bookmark · DDL auto-commits

Units III–V

Topic	Remember
Exception kinds	Predefined · non-predefined (PRAGMA EXCEPTION_INIT) · user-defined (RAISE)
Common predefined	NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, DUP_VAL_ON_INDEX, INVALID_NUMBER, VALUE_ERROR
RAISE_APPLICATION_ERROR	Number -20000 to -20999 plus a message
SQLCODE / SQLERRM	Error number / error text; OTHERS handler always last
Procedure / function	Action, called as a statement / returns one value, used in expressions and SELECT
Parameter modes	IN (default, read-only) · OUT (returned) · IN OUT (both)
Package	Specification (public declarations) + body (code, private items)
Trigger timing / level	BEFORE · AFTER · INSTEAD OF / statement · row (FOR EACH ROW)
:OLD / :NEW	INSERT: old NULL · DELETE: new NULL · UPDATE: both filled
Mutating table	ORA-04091: row trigger touched its own table; use compound trigger
Collections	Associative array (key-value) · nested table (from 1) · VARRAY (fixed limit)
Bulk operations	BULK COLLECT (read many) · FORALL (write many) · LIMIT for big data
Autonomous transaction	PRAGMA AUTONOMOUS_TRANSACTION; commits independently; use for logging
Static / dynamic SQL	Compile time / run time with EXECUTE IMMEDIATE; bind with USING
Definer / invoker rights	Owner's privileges (default) / AUTHID CURRENT_USER — caller's

Exam-day revision checklist

Tick these off — if you can **write or draw** each one from memory, you are well covered.

- [ ] SQL vs PL/SQL table; advantages of PL/SQL
- [ ] PL/SQL engine architecture diagram
- [ ] Block structure with optional / mandatory sections; nested block scope
- [ ] Variable, constant and datatype declarations; %TYPE and %ROWTYPE
- [ ] IF-ELSIF and CASE programs; the three loops with examples
- [ ] NULL handling and the three-valued logic table
- [ ] Implicit cursor attributes with an UPDATE example
- [ ] Explicit cursor life cycle with a program; the four attributes
- [ ] Cursor FOR loop and parameterized cursor
- [ ] FOR UPDATE / WHERE CURRENT OF; REF CURSOR and SYS_REFCURSOR
- [ ] COMMIT, ROLLBACK, SAVEPOINT and the ACID properties
- [ ] Three exception types with examples; propagation diagram
- [ ] RAISE_APPLICATION_ERROR, SQLCODE, SQLERRM
- [ ] Procedure with IN / OUT / IN OUT; positional vs named notation
- [ ] Function returning a value, used from SELECT; procedure vs function table
- [ ] Package specification and body; advantages of packages
- [ ] Trigger syntax, firing order diagram, :OLD / :NEW, audit trigger example
- [ ] Mutating-table error; INSTEAD OF trigger
- [ ] Records and the three collection types with methods
- [ ] BULK COLLECT, FORALL, LIMIT and why bulk is faster
- [ ] Autonomous transaction logger
- [ ] Dynamic SQL with bind variables; SQL injection and its prevention
- [ ] Definer vs invoker rights; six performance best practices
- [ ] Bank-transfer procedure from Unit V
- [ ] Prime, Fibonacci, factorial and palindrome programs

Recommended resources

Resource	Covers
Oracle Database PL/SQL Language Reference (docs.oracle.com)	Authoritative syntax for every unit
Steven Feuerstein — *Oracle PL/SQL Programming* (O'Reilly)	Units I–V, best practices and bulk processing
Ivan Bayross — *SQL, PL/SQL: The Programming Language of Oracle*	Common university textbook, many lab programs
Oracle Database Free / XE and SQL Developer (oracle.com)	Run every example yourself
geeksforgeeks.org — PL/SQL articles	Extra practice for Units I–IV

All diagrams in this book were drawn for this book. Per-topic "GFG" links open GeeksforGeeks **search** pages for extra reading. Example outputs were worked out by hand from the sample data and have **not** been run on a live Oracle instance here — run each one yourself, and cross-check university-specific slides for local terminology.